

ЛЕКЦИЯ 5



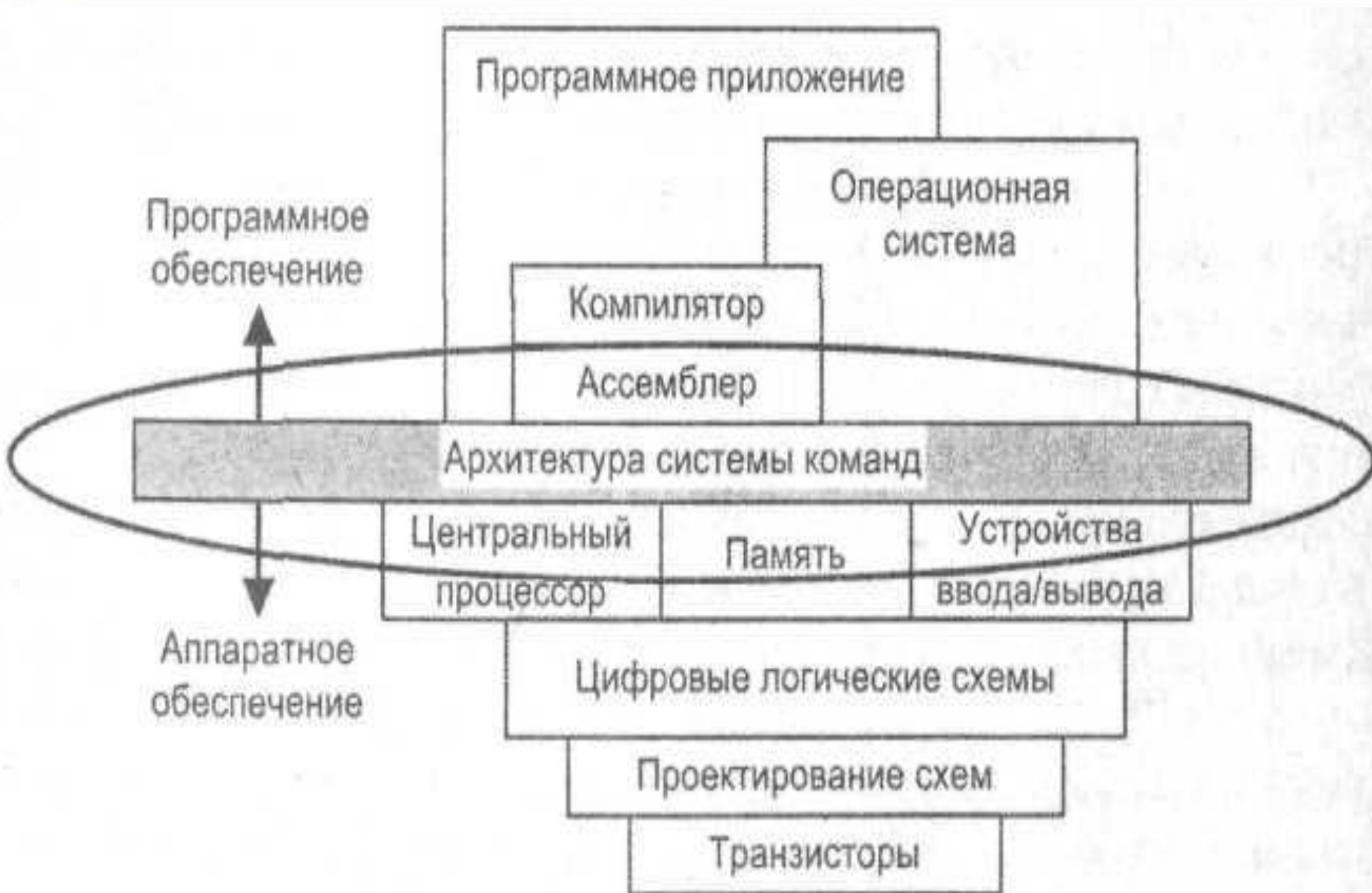
СИСТЕМЫ КОМАНД ЦЕНТРАЛЬНОГО ПРОЦЕССОРА

- Архитектуры системы команд
- Архитектуры CISC, RISC и VLIW
- Способы адресации операндов
- Примеры команд
- Форматы команд микропроцессоров



- ***Система команд вычислительной машины*** — это полный перечень команд, которые способна выполнять данная машина.
- ***Архитектура системы команд (АСК) вычислительной машины*** — это совокупность средств, которые видны и доступны программисту.

Архитектура системы команд





- Вид и форматы данных.
- Место хранения данных (помимо основной памяти).
- Методы доступа к данным.
- Операции над данными.
- Количество операндов в команде.
- Способ определения адреса очередной команды.
- Способ кодирования команды.



Архитектура системы команд

Аккумуляторная архитектура, 1950

Стековая архитектура, 1963-66

Регистровая архитектура, 1964

CISC – архитектура
с полным набором
команд, 1977-80

Архитектура с выделенным
доступом к памяти, 1963-76

RISC – архитектура с
сокращённым набором
команд, 1987

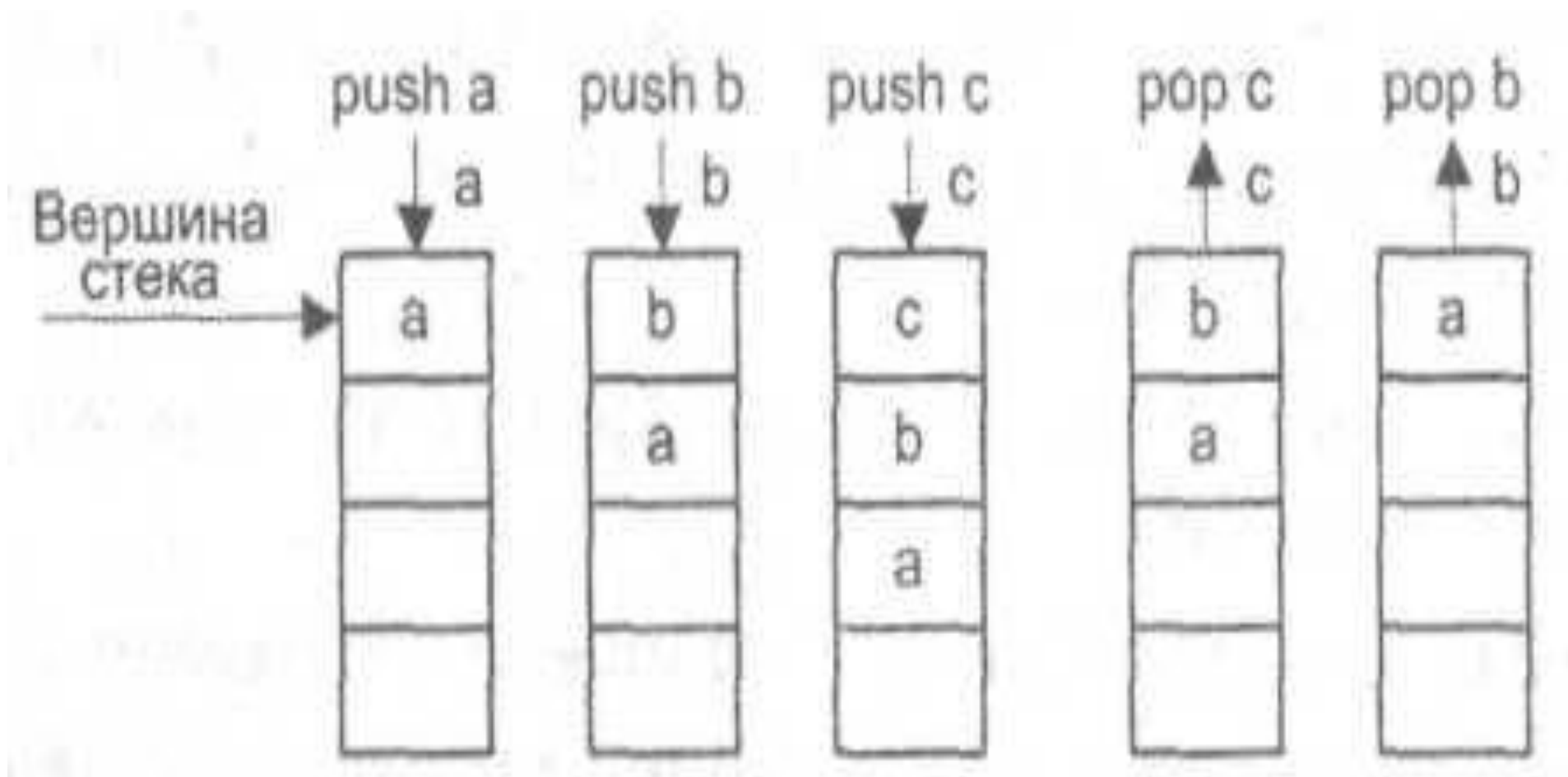
VLIW – архитектура с
командами сверхбольшой
длины, конец 1990х

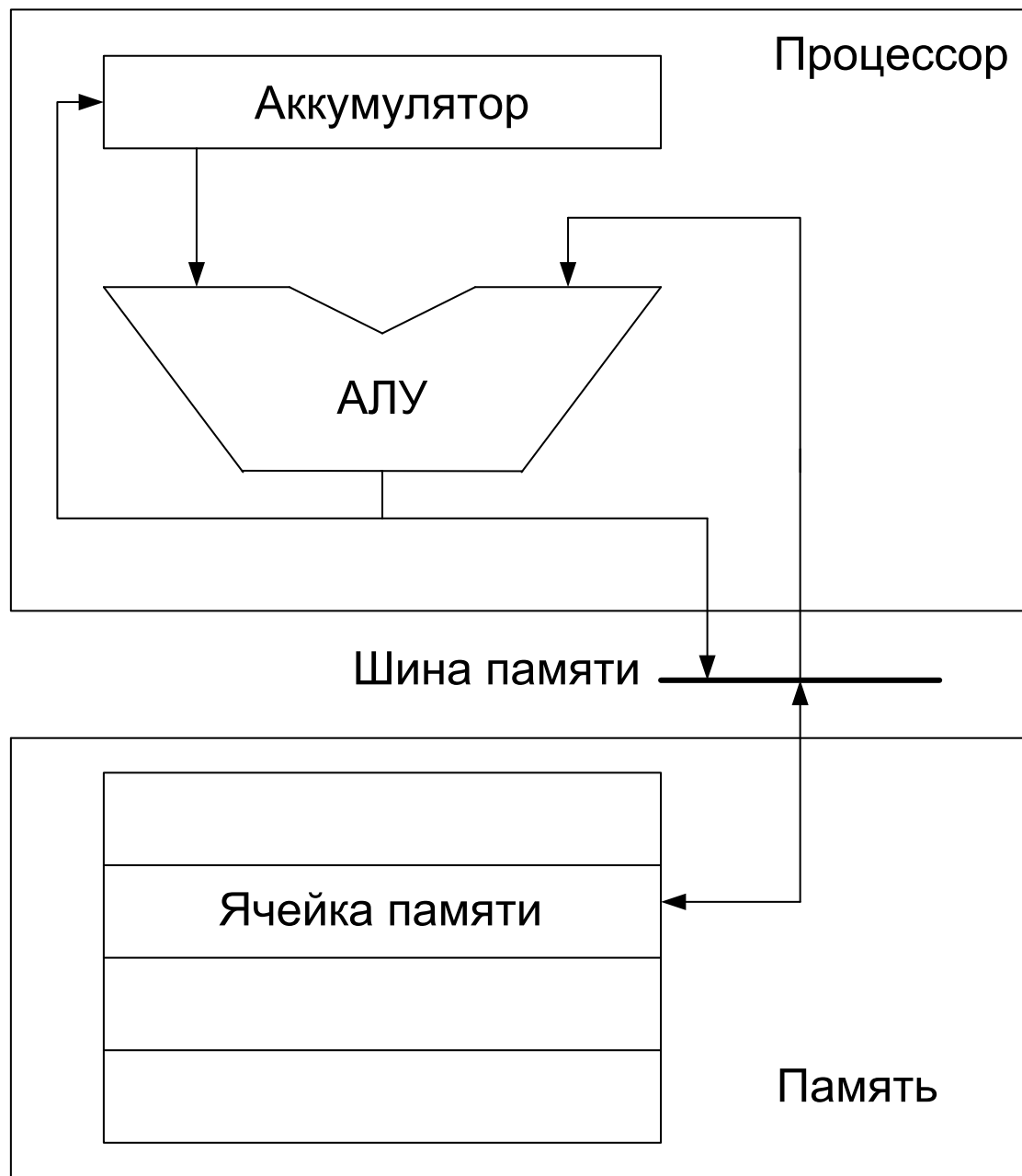
ROSC – архитектура с
безоперандным набором
команд, 2001



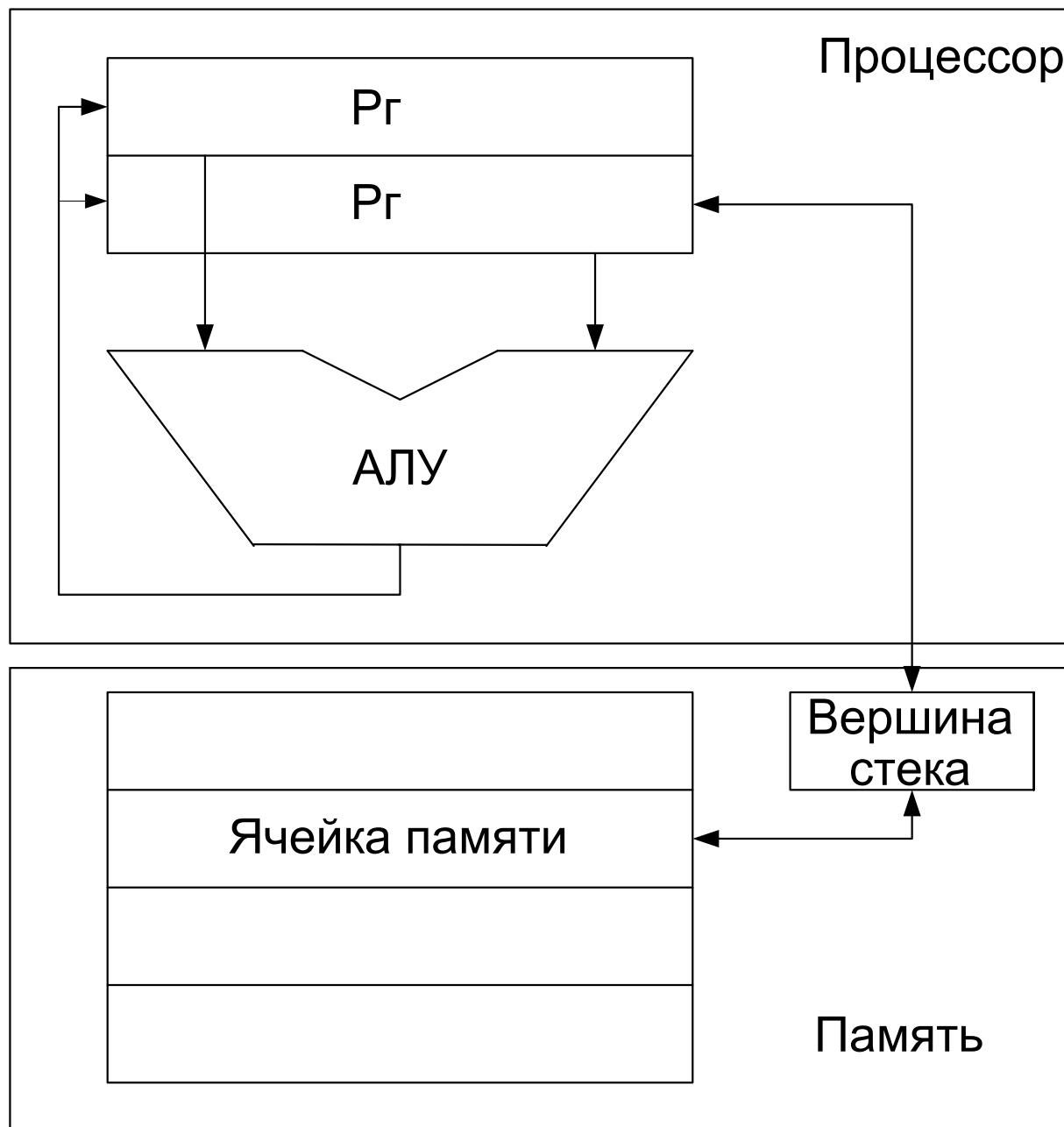
- По типу выполняемых операций (общего назначения, специализированные, дополненной системой команд).
- По месту хранения операндов (тип адресуемой памяти – стековая, аккумуляторная, регистровая и с выделенным доступом к памяти).
- По составу и сложности команд (CISC, RISC и VLIW).

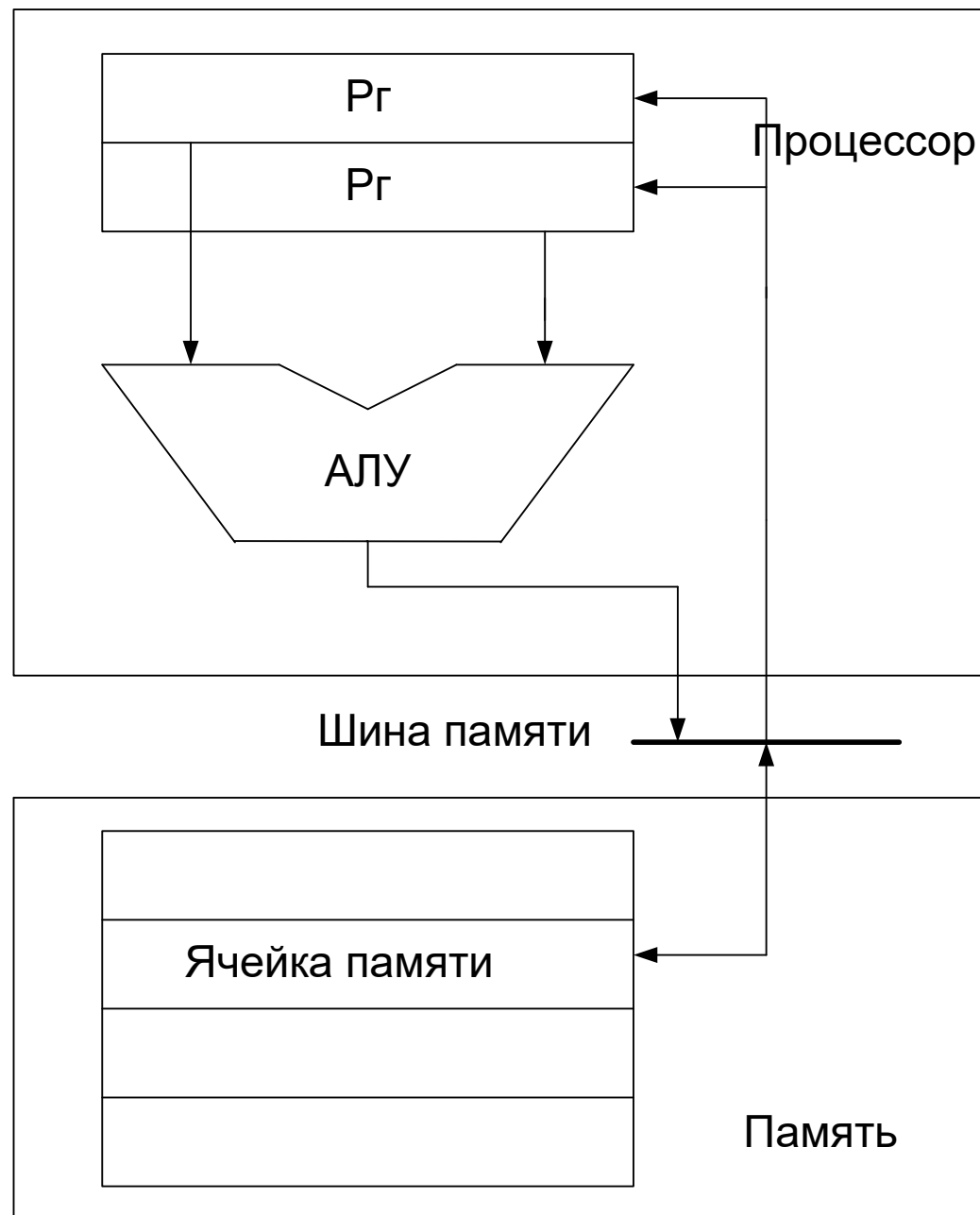
Принцип действия стековой памяти



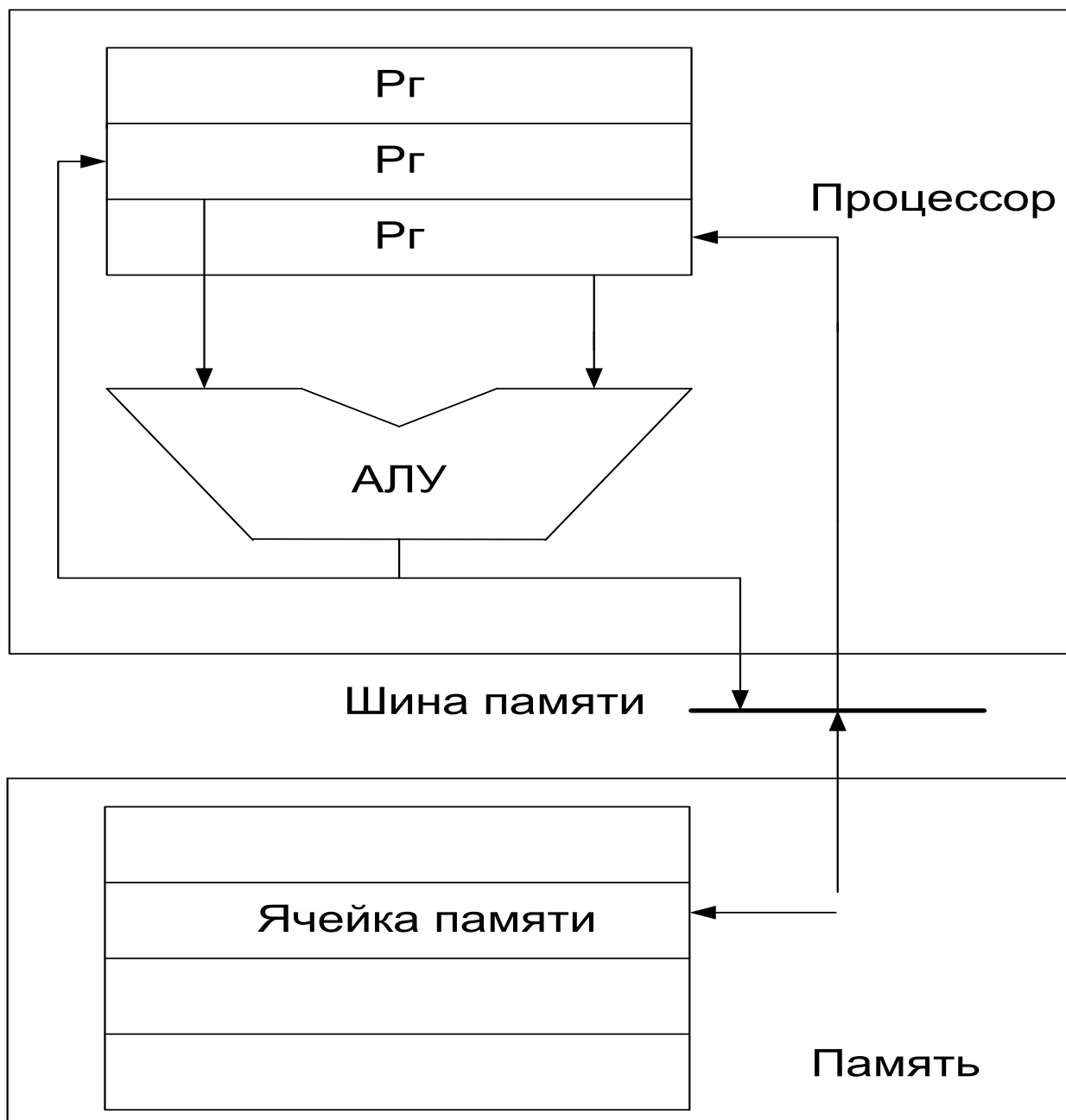


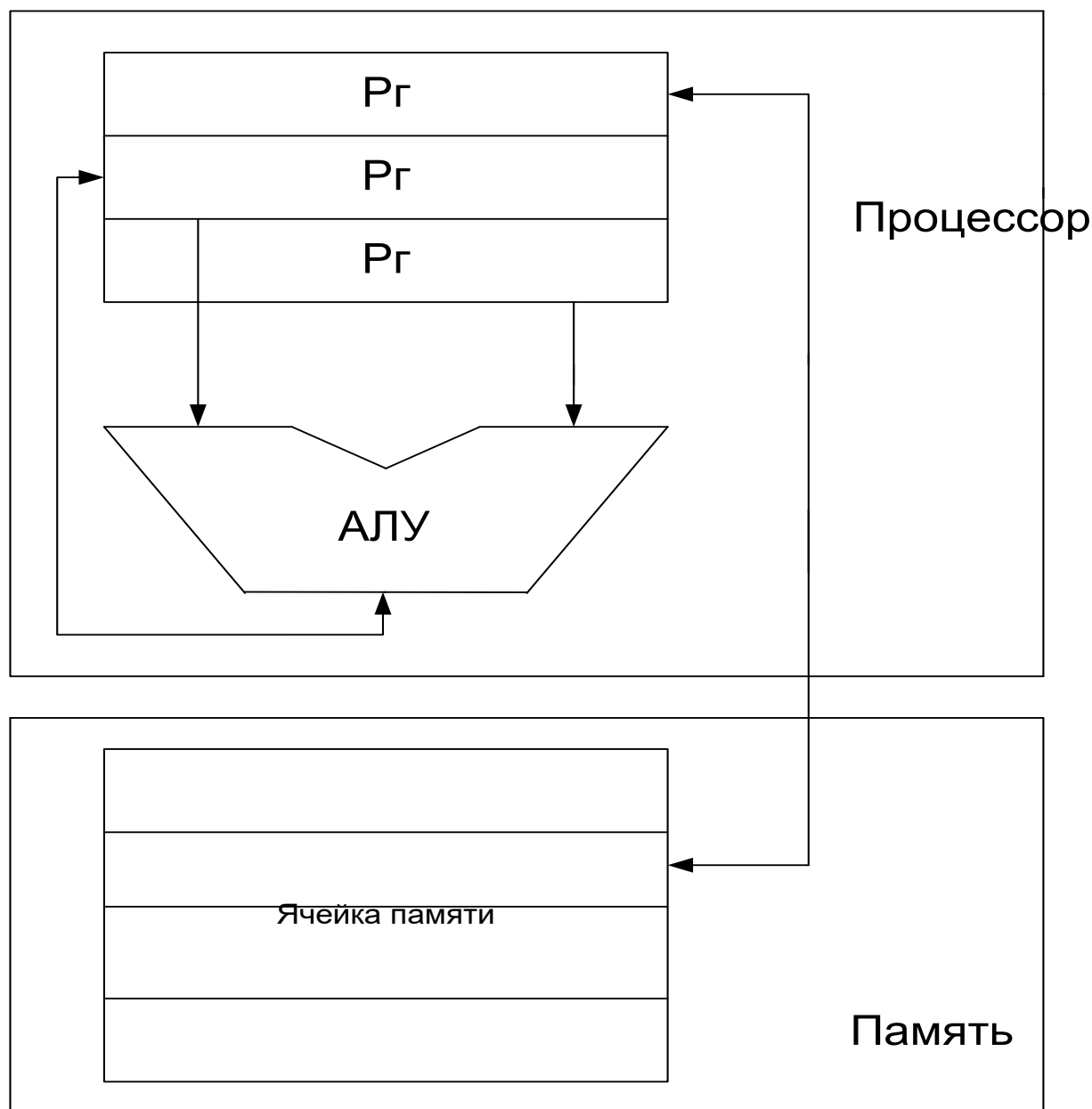
Архитектура ВМ на базе стека





Архитектура ВМ на базе РОН (регистр-память)





Тип выполняемых операций



Вариант	Достоинства	Недостатки
Регистр-регистр (0,3)	Простота реализации, фиксированная длина команд, простая модель формирования объектного кода, возможность выполнения всех команд за одинаковое количество тактов	Большая длина объектного кода, часть разрядов в коротких командах не используется
Регистр-память (1,2)	Данные могут быть доступны без загрузки в регистры процессора, простота кодирования команд, объектный код получается достаточно компактным	Потеря одного из операндов при записи результата, длинное поле адреса памяти в коде команды сокращает место под номер регистра, что ограничивает общее число РОН
Память-память (3,3)	Компактность объектного кода, малая потребность в регистрах для хранения промежуточных данных	Разнообразие форматов команд и времени их исполнения, низкое быстродействие из-за обращения к памяти



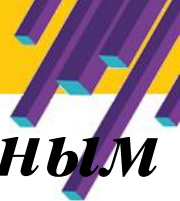
- Для доступа к памяти используются две специальные команды:
 - ***Load*** – загрузка, считывание значения из основной памяти и занесение его в регистр;
 - ***Store*** – сохранение, пересылка информации в обратном направлении.

- Операнды всех команд обработки могут находиться только в регистрах.

- Главное достоинство архитектуры – простота декодирования и исполнения команд



- Архитектура с полным набором команд: CISC (Complex Instruction Set Computer);
- Архитектура с сокращённым набором команд: RISC (Reduced Instruction Set Computer);
- Архитектура с командными словами сверхбольшой длины: VLIW (Very Long Instruction Word).



CISC – complex instruction set computer, компьютер с полным набором команд. Характерные свойства:

- нефиксированное значение длины команды;
- арифметические действия кодируются в одной команде;
- небольшое число регистров, каждый из которых выполняет строго определённую функцию.

Недостатки CISC архитектуры:

- высокая стоимость аппаратной части;
- сложности с распараллеливанием вычислений;
- **80% времени используются 20% общего набора команд.**

Пример CISC – ЦП Intel x86:

- более 200 команд;
- длина команды – 1-15 байтов;
- более 10 способов адресации.



RISC – reduced instruction set computer, компьютер с сокращённым набором команд. Быстродействие увеличивается за счёт упрощения инструкций: простое декодирование, малое время выполнения.

- Около 100 команд
- Фиксированная длина команд (4 байта против 1-15 у CISC)
- Часто ограничиваются регистровой и непосредственной адресацией
- Не менее 32 РОН (против 8-16 в CISC)
- Одна команда в 1 такт (против 10 тактов у CISC)

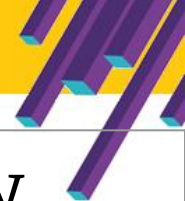


- выполнение команд (не менее 75%) за один машинный цикл;
- длина всех команд равна машинному слову и допускает унифицированную поточную обработку команд;
- малое число команд (не более 128), форматов команд (не более 4), способов адресации (не более 4);
- доступ к памяти только посредством команд «Чтение» и «Запись»;
- все команды, за исключением «Чтения» и «Записи», используют внутрипроцессорные межрегистровые пересылки;
- устройство управления схемного типа;
- большой регистровый файл (не менее 32, иногда > 500).



VLIW (Very Long Instruction Word) – архитектура с командными словами сверхбольшой длины.

Концепция VLIW базируется на RISC-архитектуре, где несколько простых RISC-команд объединяются **компилятором** в одну сверхдлинную команду и выполняются параллельно, как правило за 1 такт.



Характеристика	CISC	RISC и VLIW
Длина команды	Варьируется	Единая
Расположение нулей в команде	Варьируется	Неизменное
Количество регистров	Несколько (часто специализированных)	Много регистров общего назначения
Доступ к памяти	Может выполняться как часть команд различных типов	Выполняется только специальными командами



1. Проводит дешифрацию кодов команд.
2. Производит пересылку данных между РОН, ЦП и ОП.
3. Осуществляет преобразование данных разных форматов.
4. Помогает АЛУ осуществлять многошаговые инструкции.
5. Выполнение сегментации памяти.
6. Производит страничные преобразования.
7. Осуществляет управление шиной компьютера.
8. Управляет разделением потоков в SMP-системах и взаимодействием с сопроцессорами.
9. Формирует сигналы управления АЛУ и системными регистрами.
10. Реализовывает конвейерные вычисления и инструкции, организацию доступа SIMD.
11. Проводит политику защиты выполняемых процессов путём назначения программам разных уровней привилегий и приоритетов.
12. Управляет контроллерами ввода-вывода.
13. Эмулирует другие процессоры или работает в «режиме совместимости».

CISC	RISC
Полностью	Полностью
Полностью	Частично
Полностью	Отсутствуют



- Команды пересылки и загрузки данных.
- Команды обработки данных:
 - арифметические;
 - логические.
- Команды ввода/вывода.
- Команды управления.
- Системные команды.



Безусловный переход (ветвление, branch)	Загружает в счётчик команд новое содержимое, являющееся адресом следующей выполняемой команды
Вызов подпрограммы	Производится путём безусловной передачи управления с сохранением адреса возврата управления
Условный переход (conditional branch)	Производит загрузку в счётчик команд нового содержимого, если выполняются определённые условия
Команды организации программных циклов	Условный переход в зависимости от значения содержимого заданного регистра, который используется как счётчик циклов
Команды прерывания	Переход к одной из программ обслуживания исключений и прерываний
Команды изменения признаков	Запись-чтение содержимого регистра состояния, в котором хранятся признаки, а также изменение значений отдельных признаков
Команды управления процессором	Команды останова, отсутствия операции и ряд команд, определяющих режим работы процессора или его отдельных блоков



- Длина команды
- Разрядность полей команды
- Количество адресов в команде
- Выбор адресности команд
- Способы адресации операндов
- Способы адресации в командах управления потоком команд
- Система операций

Варианты адресности команд

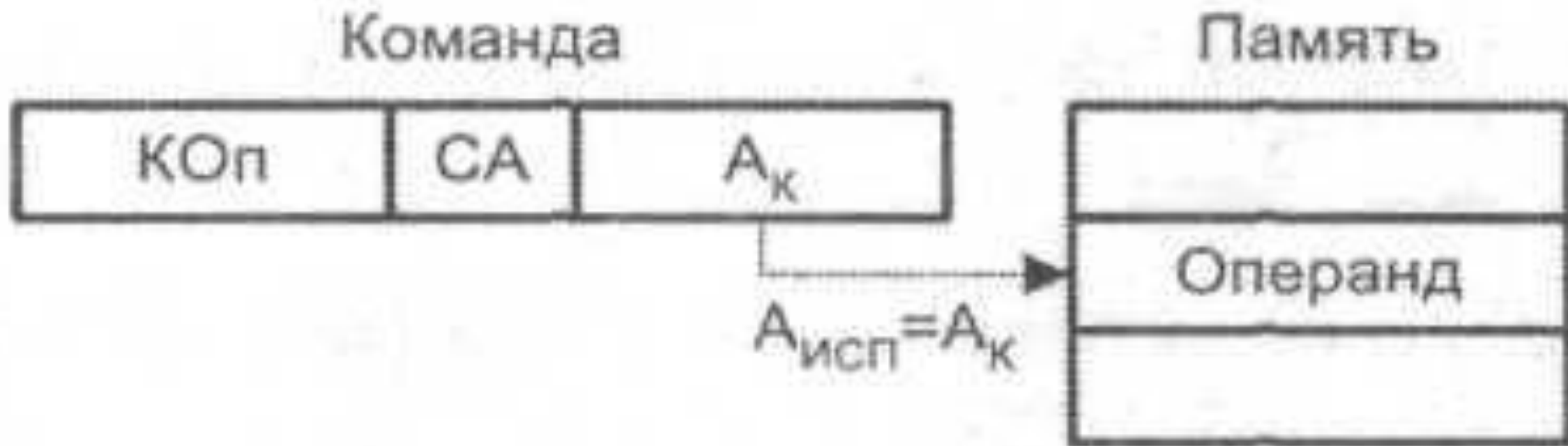
Одноадресные	А1 – либо адрес ячейки (регистра), в которой хранится один из операндов, либо адрес ячейки (регистра) для записи результата
Двухадресные	А1 – это адрес ячейки (регистра), где хранится первый операнд, и куда будет записан результат; А2 – это адрес ячейки (регистра), где хранится второй операнд
Трёхадресные	А2 и А3 – адреса ячеек (регистров), где расположены первые два операнда; А1 – адрес ячейки (регистра) для записи результата
Безадресные	Содержит только код операции, а информация для неё помещается в predetermined регистры
Комбинированные	Один или более адресов адресной части предназначены для размещения данных (непосредственный операнд)

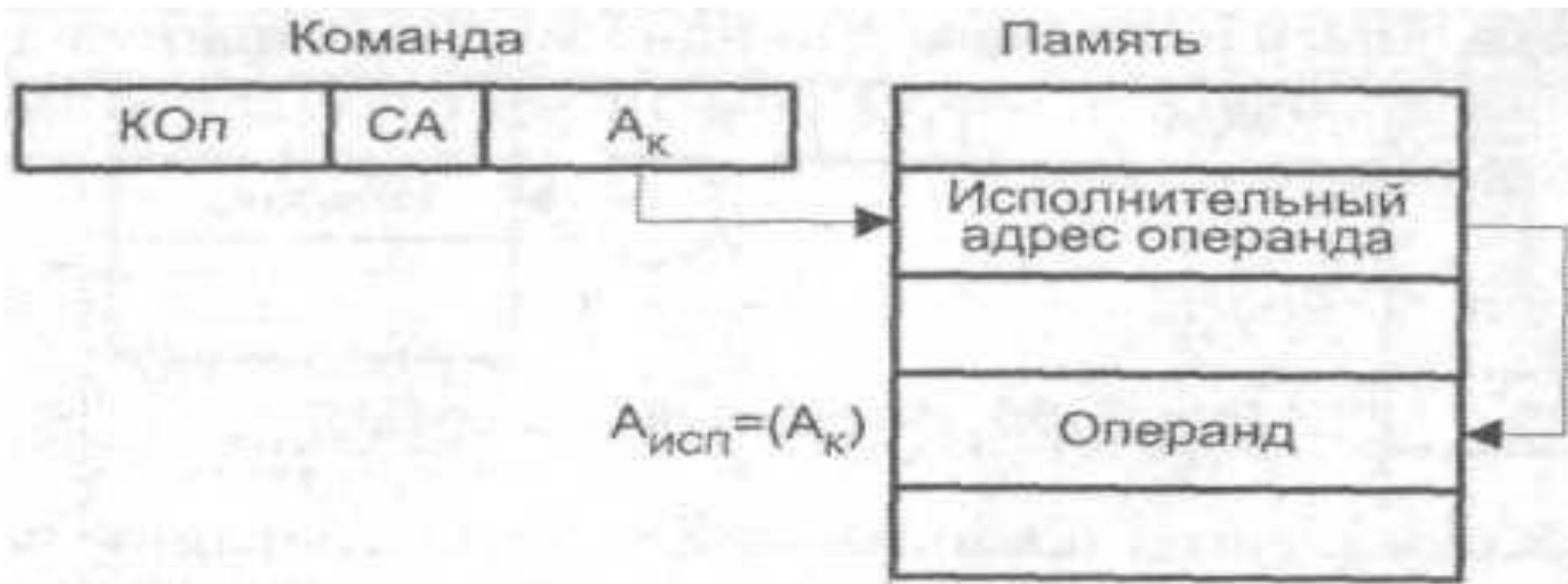


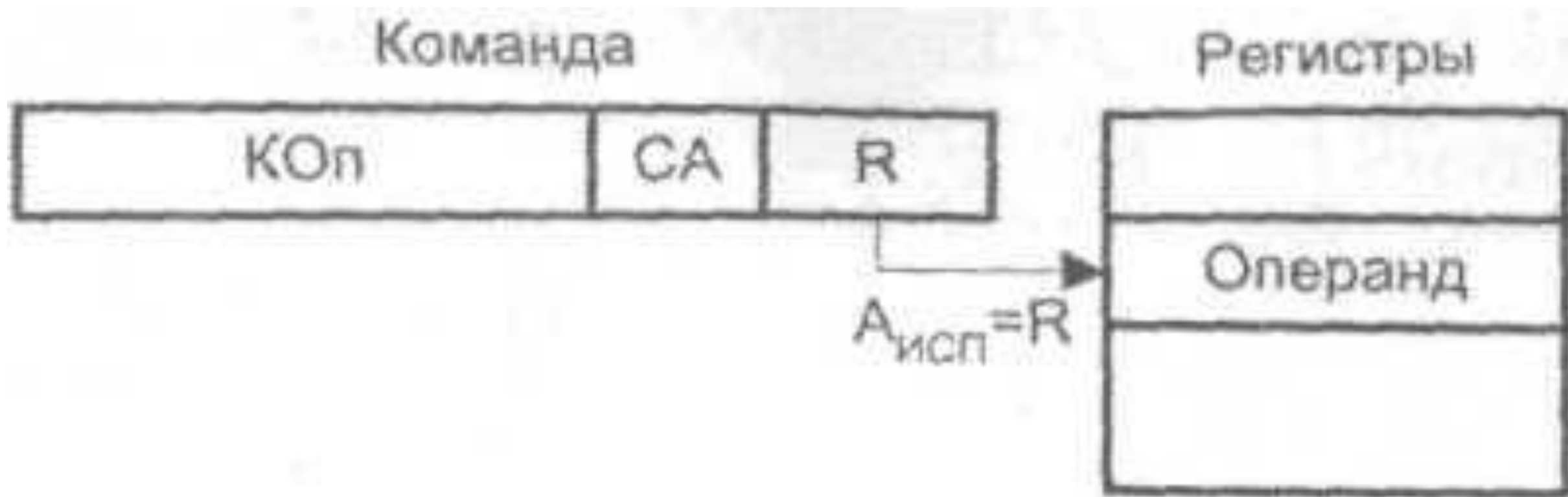
- **Прямая:** операнд – в ячейке памяти, адрес которой задаётся числом, указанным в команде.
- **Регистровая:** операнд – в регистре данных или адреса.
- **Косвенно-регистровая:** операнд – в ячейке памяти, адресуемой содержимым регистра адреса.
- **Косвенно-регистровая со смещением:** операнд – в ячейке памяти, адрес которой – сумма адресного базового регистра и смещения.
- **Косвенно-регистровая с базовым смещением и индексированием:** операнд – в ячейке памяти, адрес которой – сумма адресного базового регистра, индексного регистра (возможно масштабированного) и смещения.
- **Относительная:** операнд – в ячейке памяти, адрес которой – сумма текущего содержимого счётчика команд и смещения.
- **Непосредственная:** операнд – число в команде.

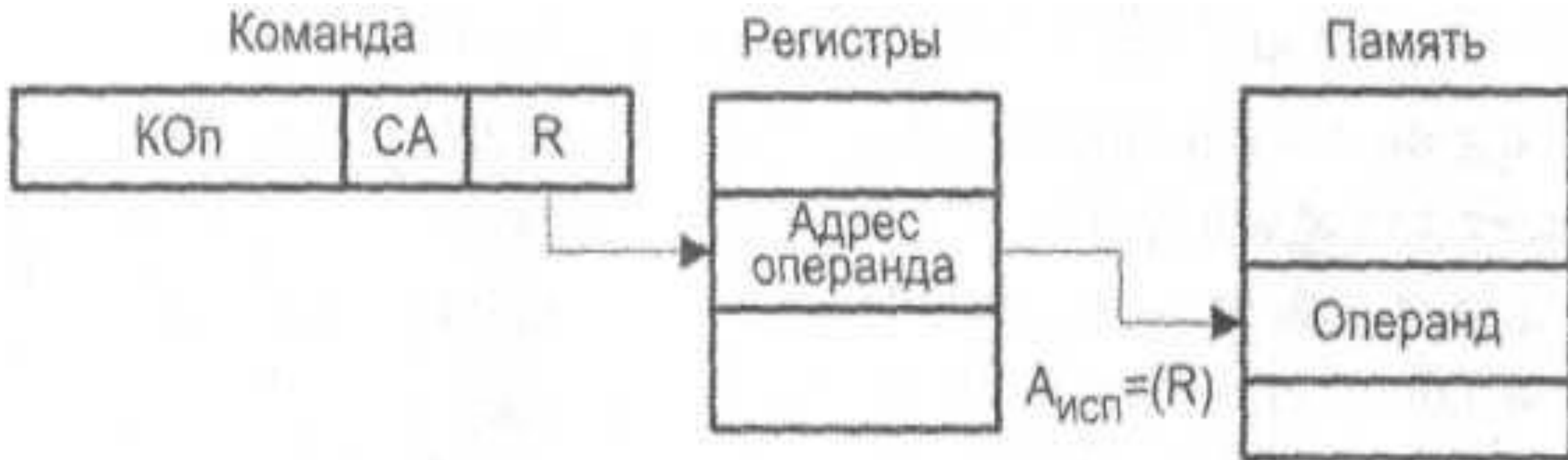


Непосредственная	<code>mov eax, 1234567h</code>
Регистровая	<code>mov eax, ecx</code>
Прямая (абсолютная)	<code>mov eax, [3456789h]</code>
Косвенно-регистровая	<code>mov eax, [ecx]</code>
Базовая/индексная со смещением	<code>mov eax, [ecx]+1200</code>
Базовая индексная со смещением	<code>mov eax, [ecx][edx]+10</code>
Индексная с масштабированием и смещением	<code>mov eax, [esi*4]+40h</code>
Базовая индексная с масштабированием	<code>mov eax, [edx][ecx*8]</code>
Базовая индексная с масштабированием и смещением	<code>mov eax, [ebx][edi*2]+3</code>

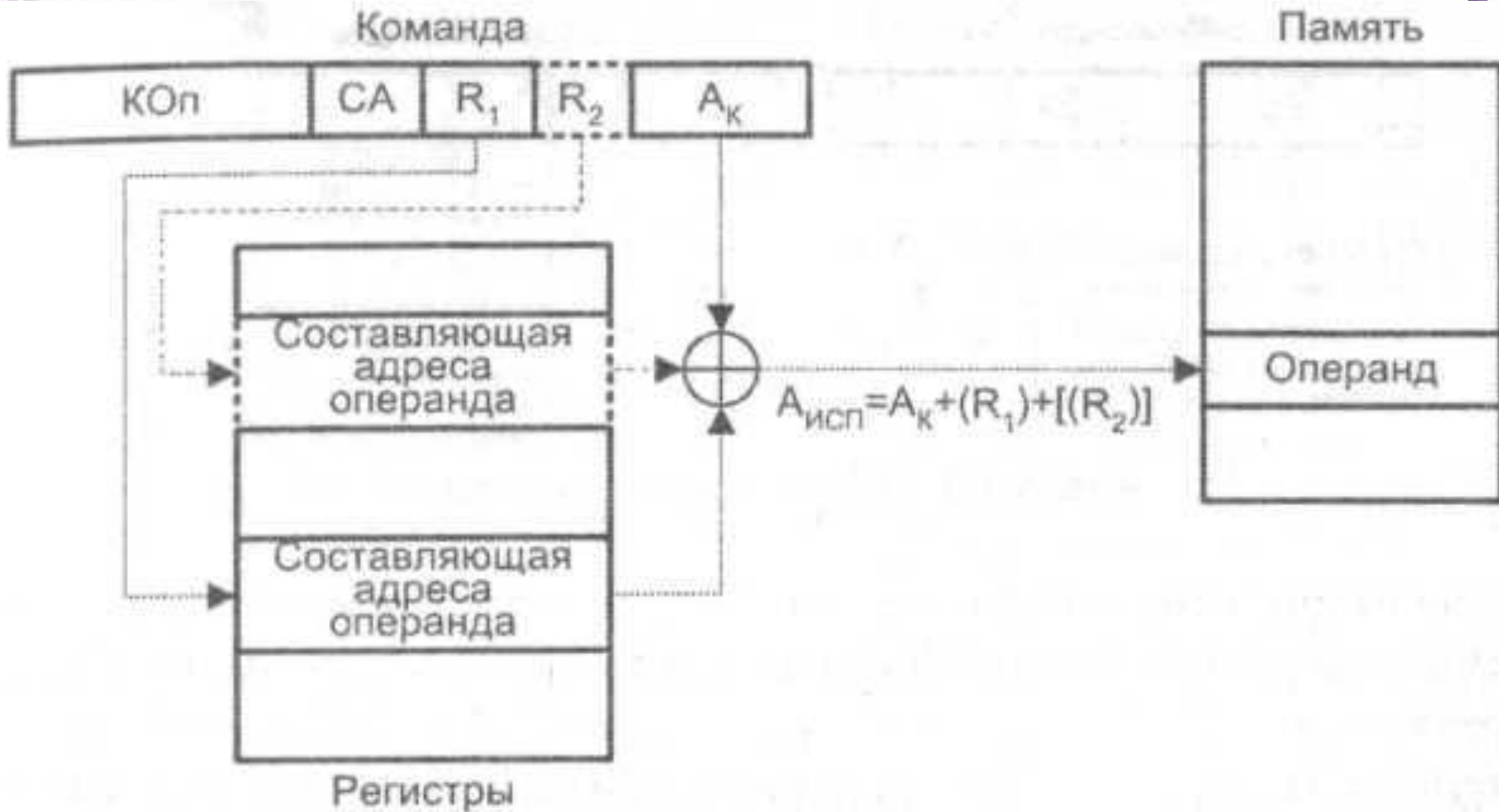




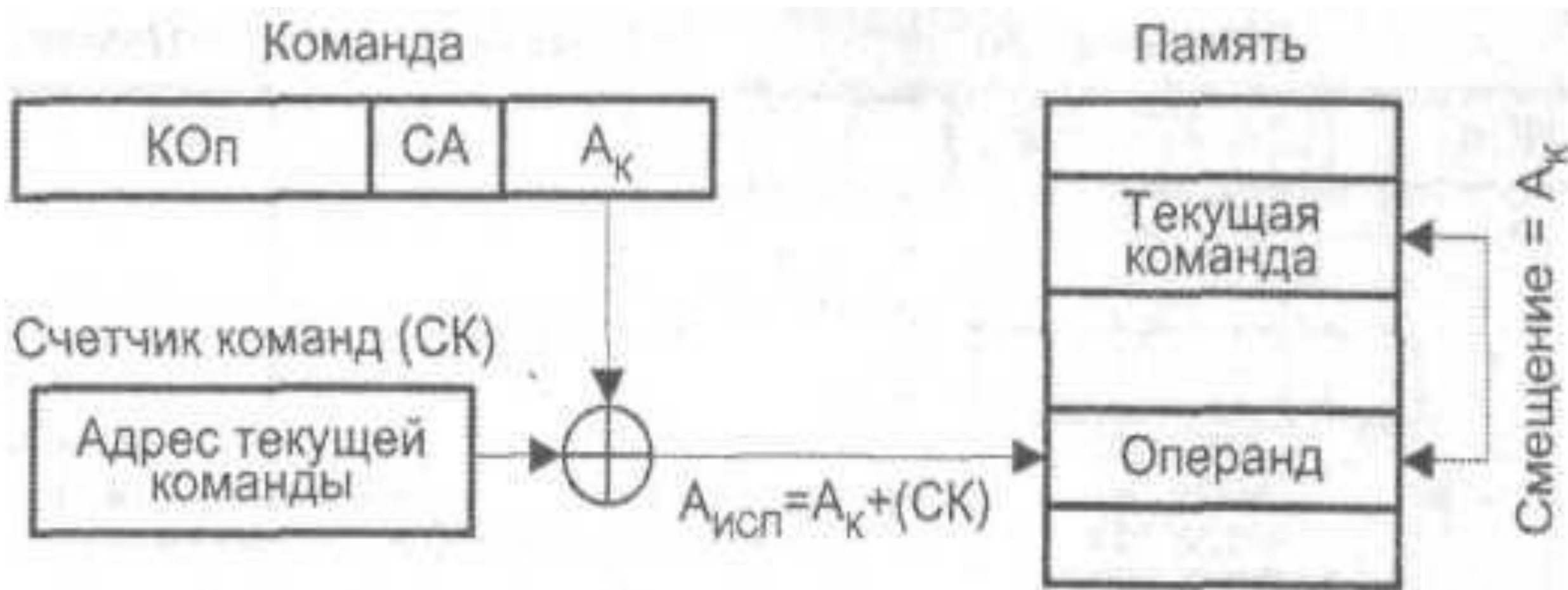


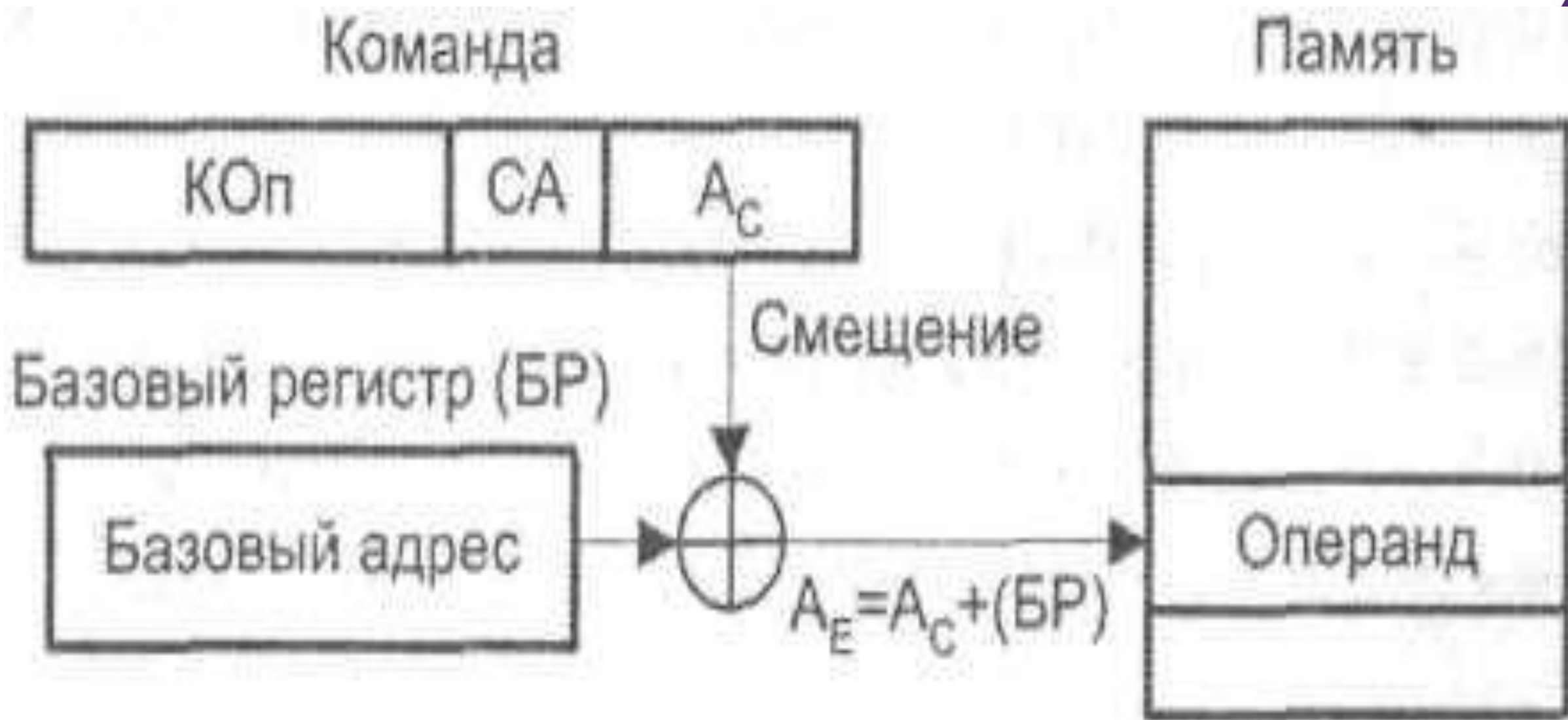


Адресация со смещением

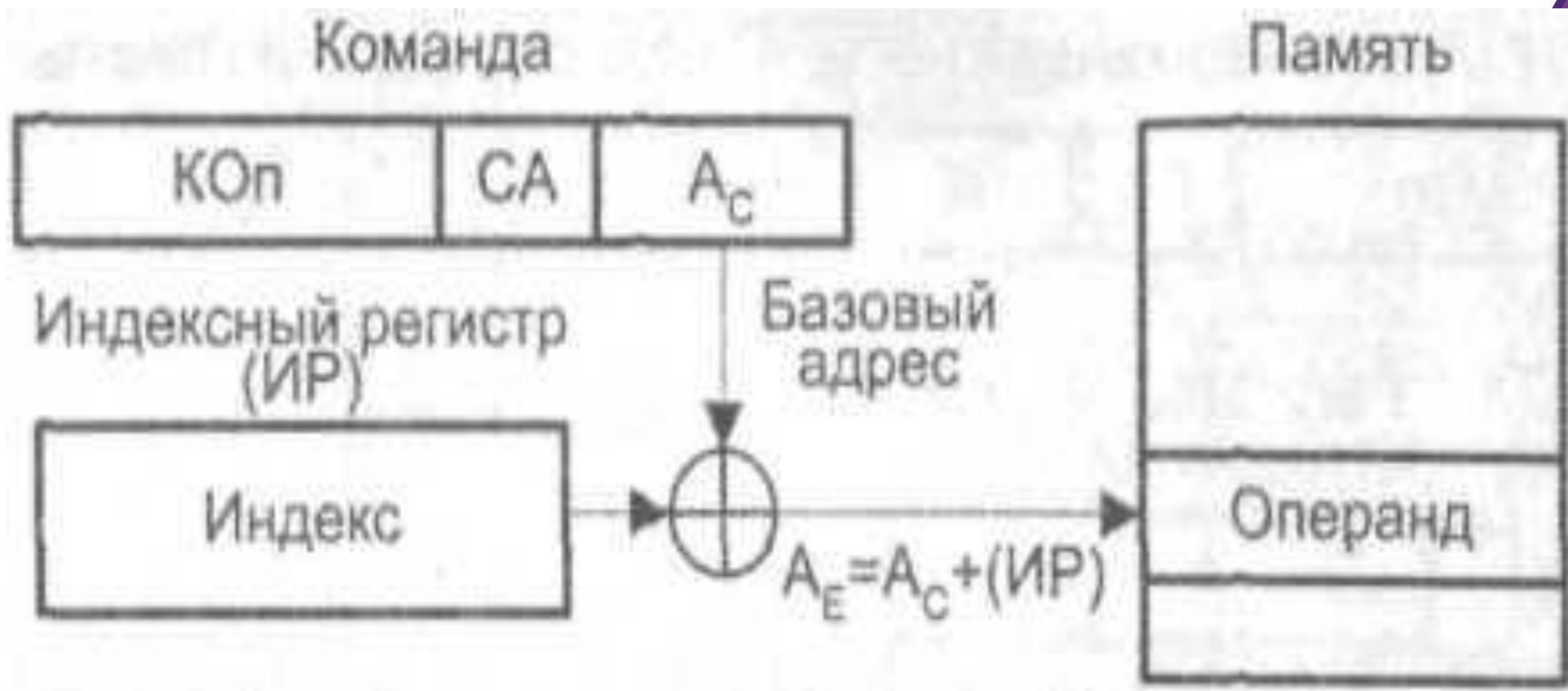


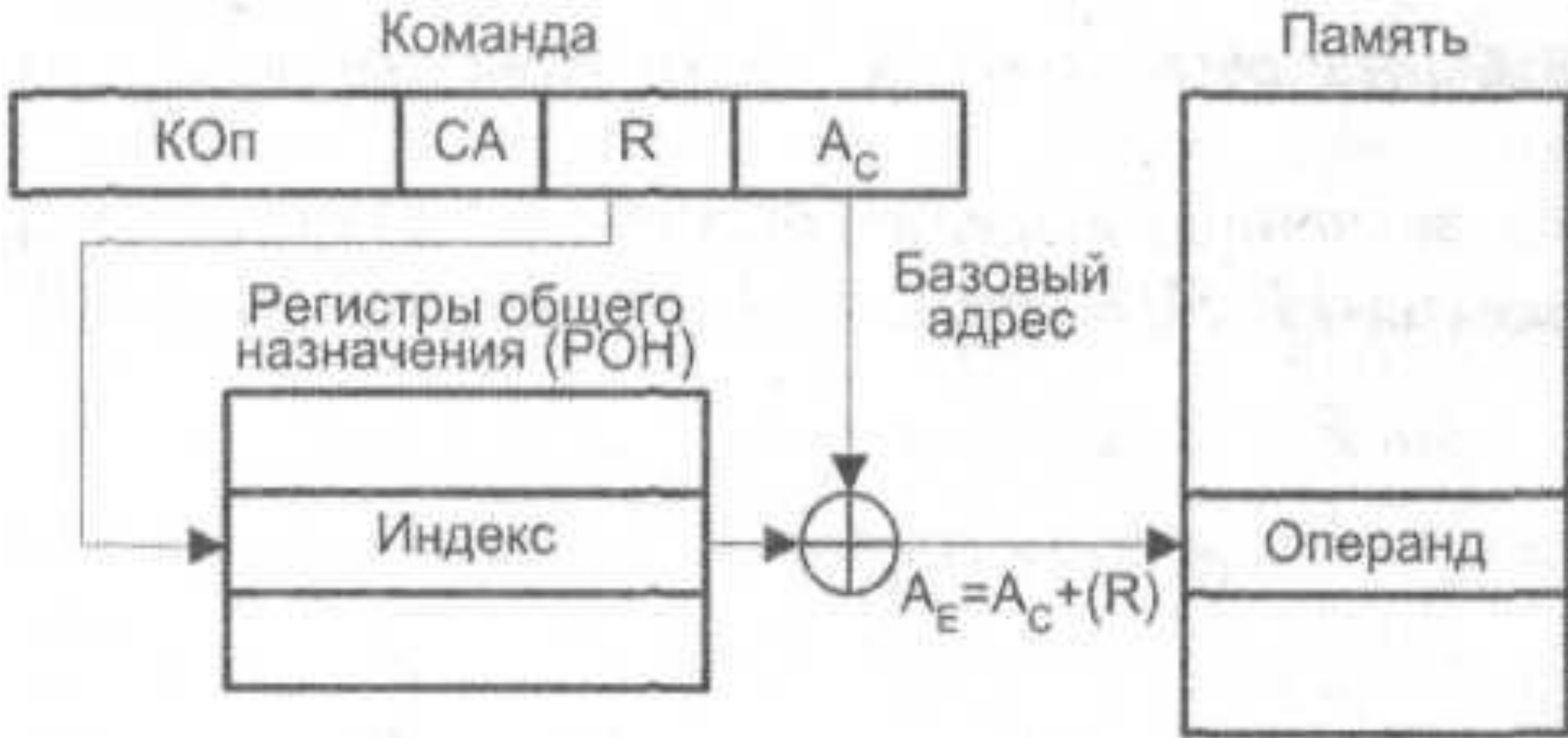
Относительная адресация

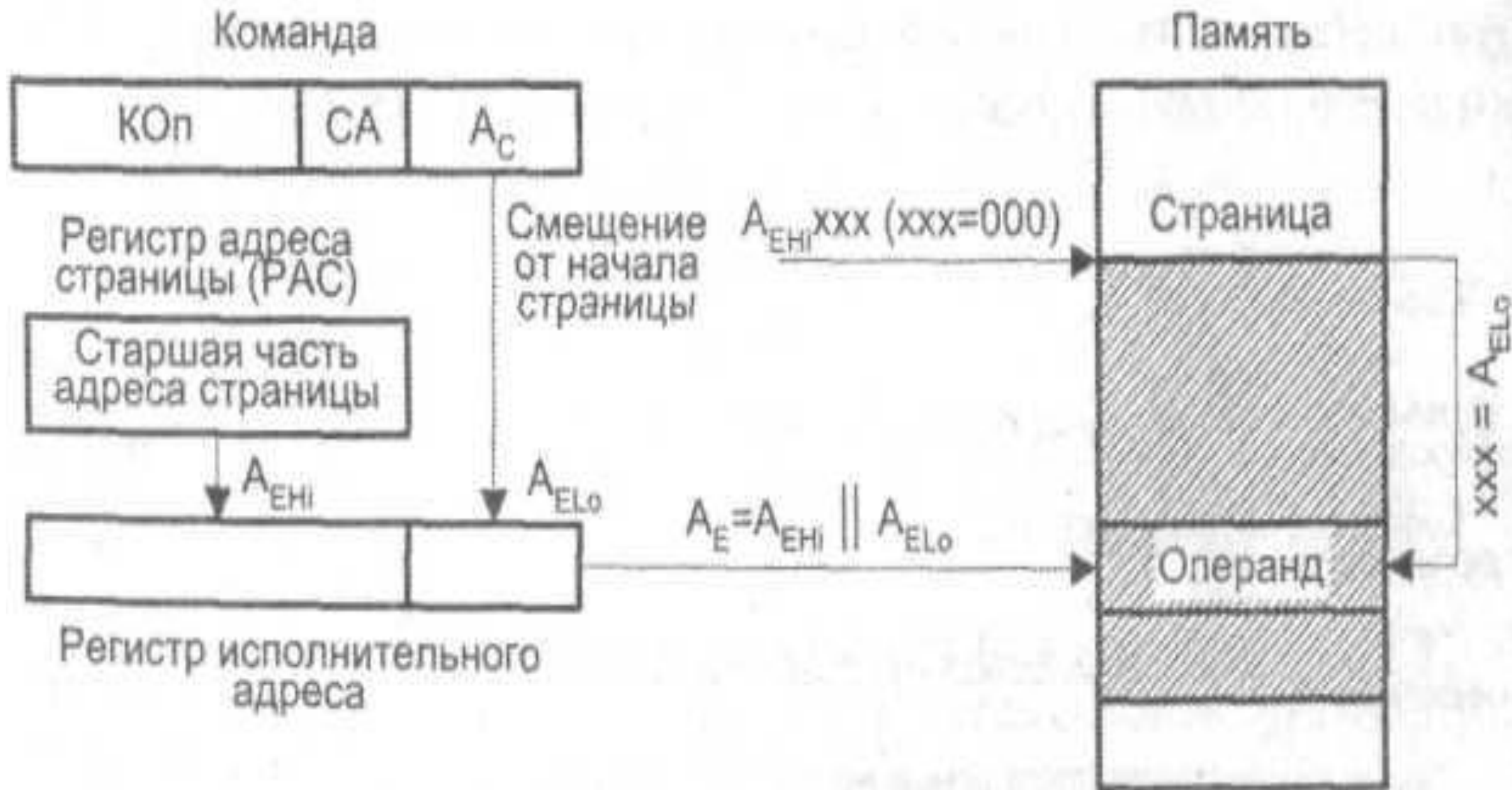














Микроконтроллер ATmega32 поддерживает режимы адресации для доступа к памяти программ (флэш-памяти) и памяти данных (SRAM, регистровому файлу, памяти ввода-вывода).

На следующих рисунках ОР означает часть кода операции командного слова. Для упрощения не на всех рисунках показано точное расположение битов адресации.

В обобщённом виде абстрактные термины RAMEND и FLASHEND использовались для обозначения самой высокой позиции в пространстве данных и программы соответственно.



Регистровая, один регистр

Регистровая, два регистра

Регистровая адресация портов ввода-вывода

Прямая адресация данных

Косвенная адресация данных

Косвенная адресация данных со смещением

Косвенная адресация данных с предкрементом

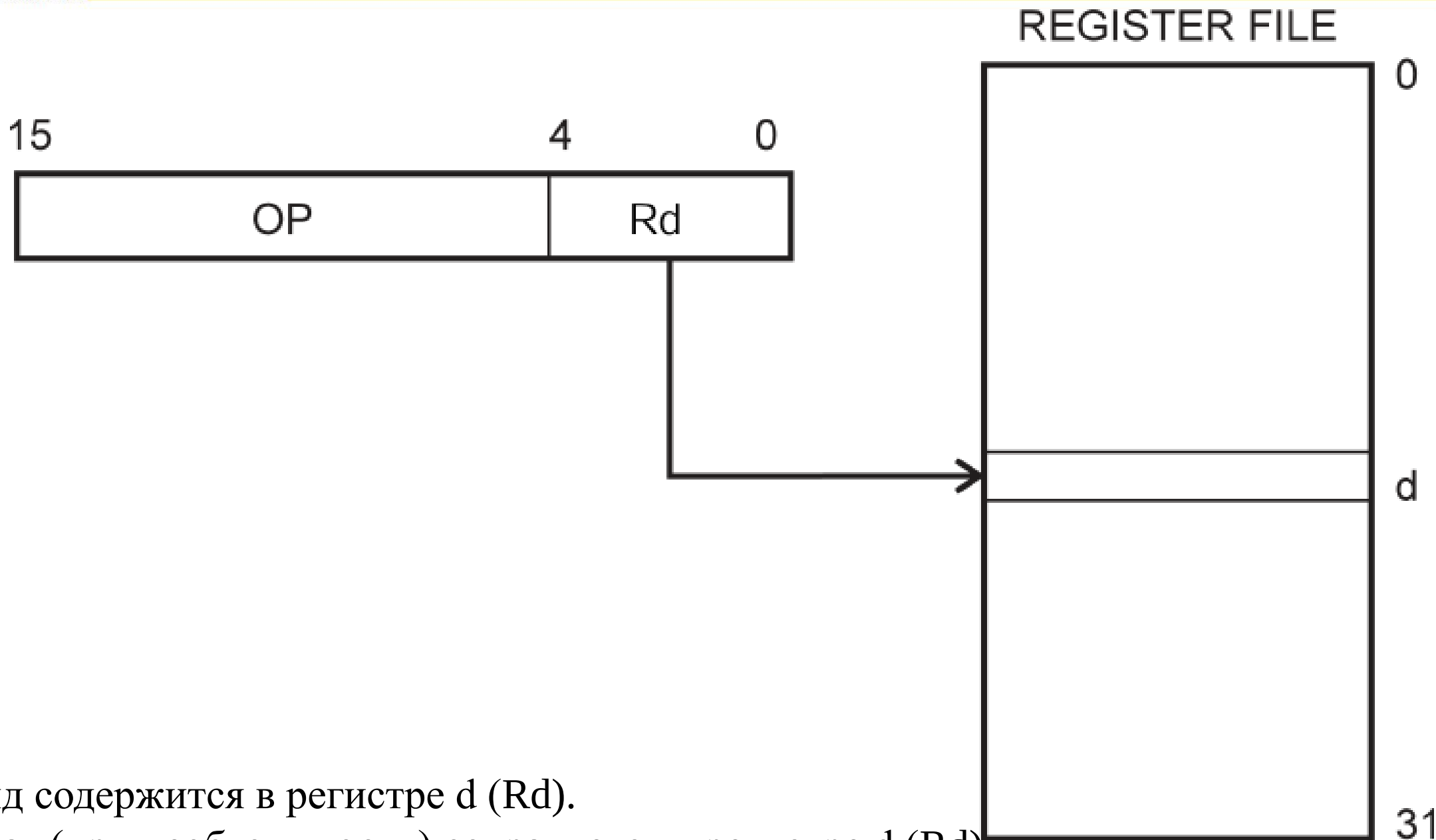
Косвенная адресация данных с постинкрементом

Прямая адресация памяти программ (JMP и CALL)

Косвенная адресация памяти программ (IJMP и ICALL)

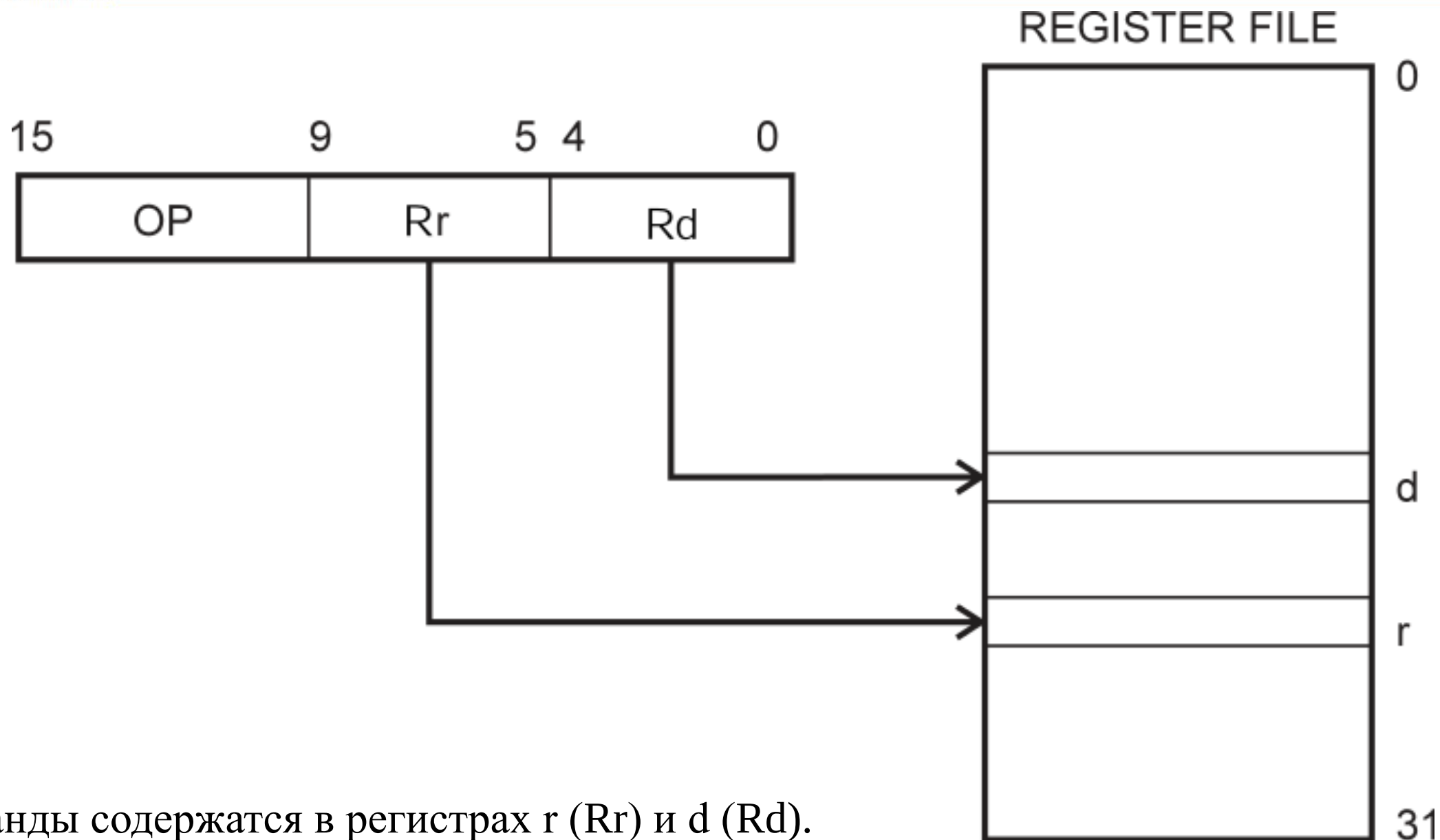
Относительная адресация памяти программ (RJMP и RCALL)

Прямая адресация памяти программ (LPM и SPM)

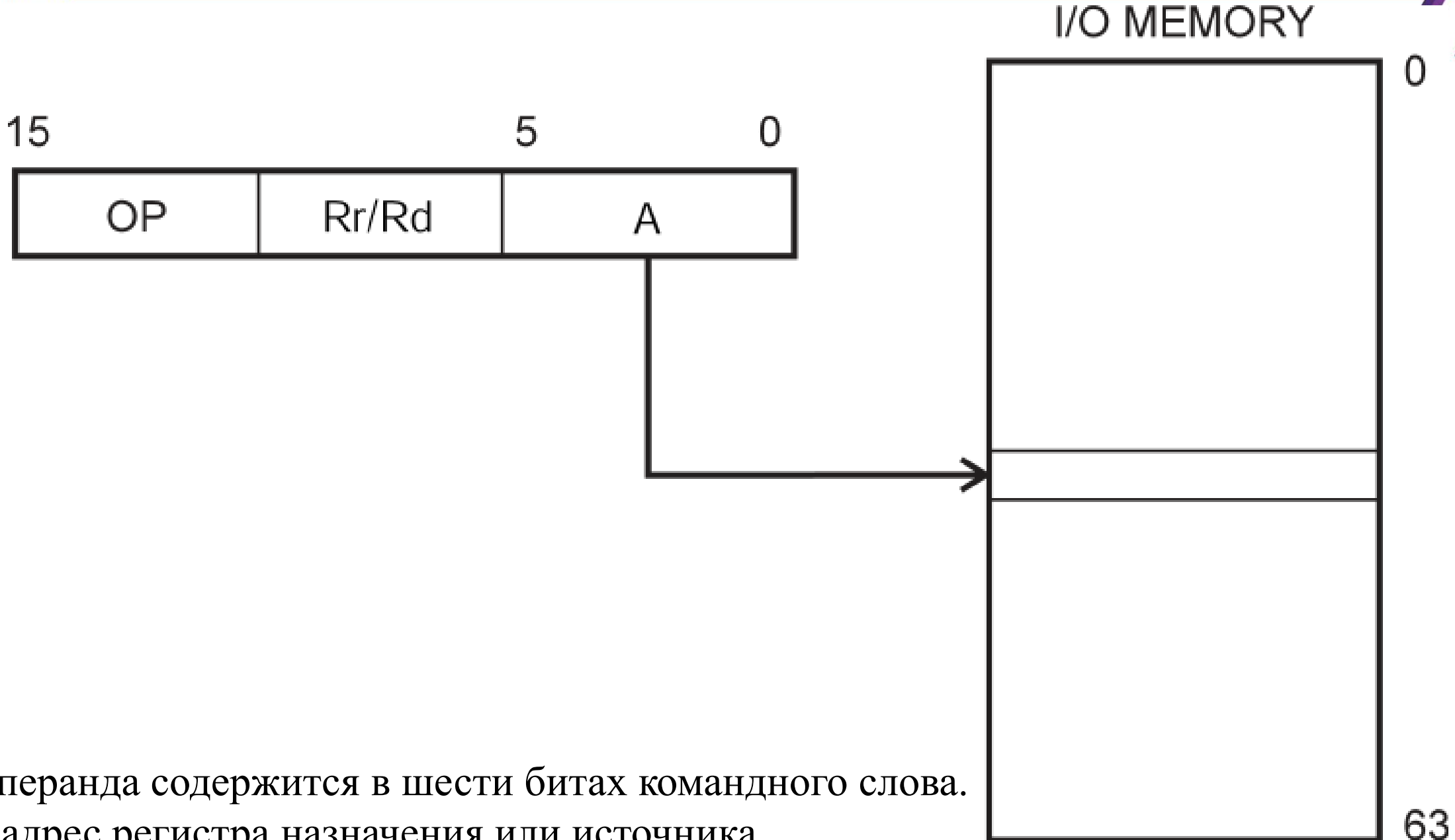


Операнд содержится в регистре d (Rd).

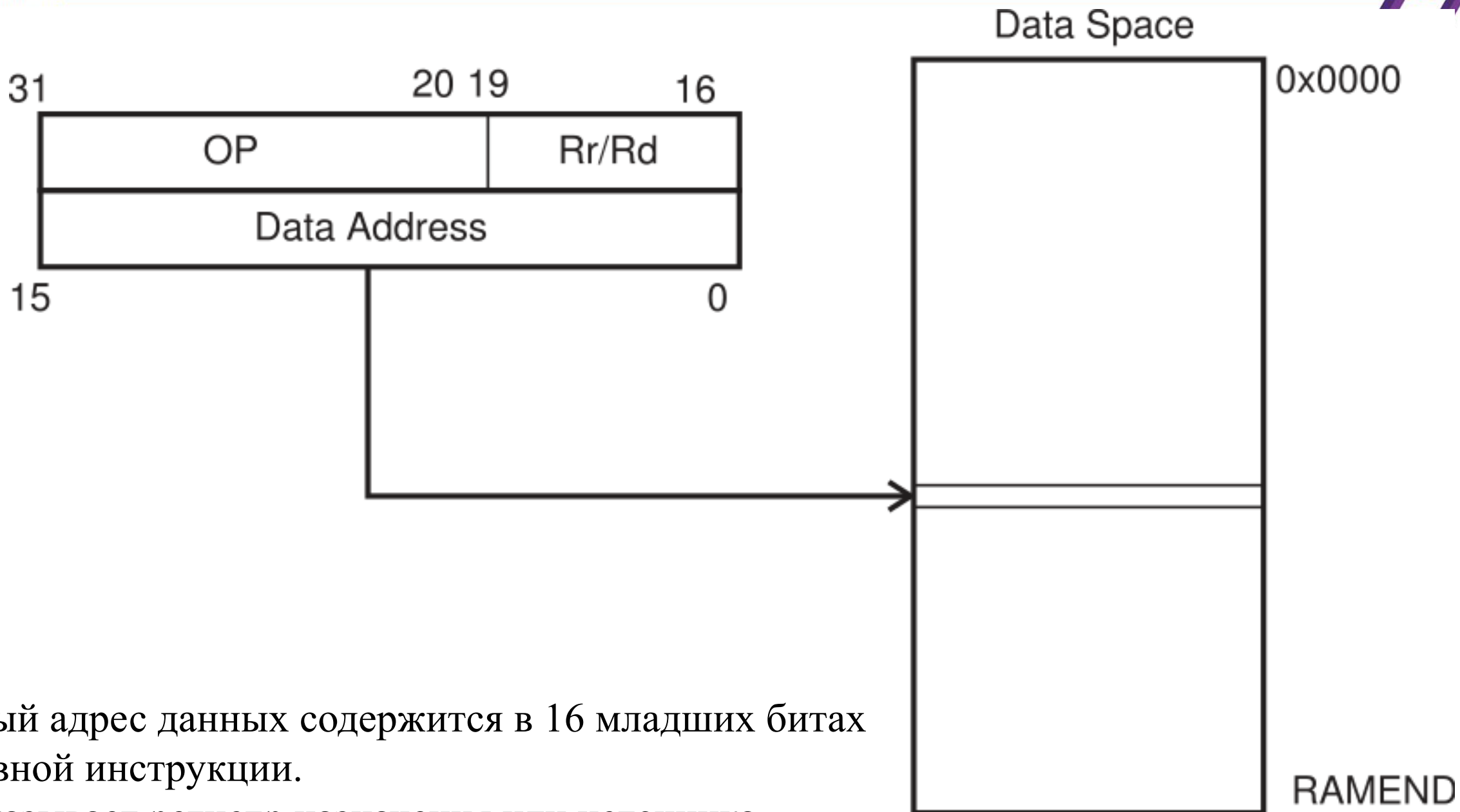
Результат (при необходимости) сохраняется в регистре d (Rd).



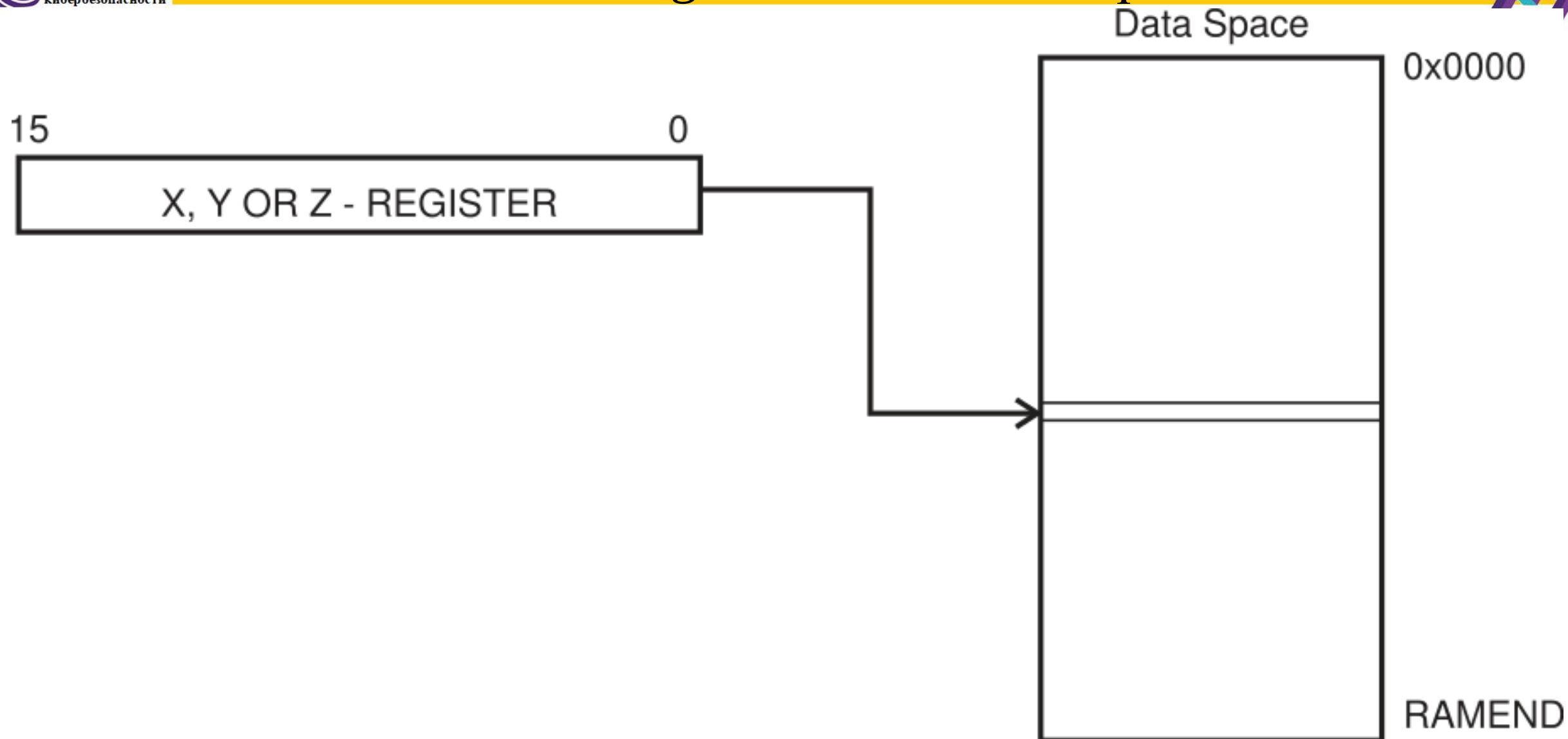
Операнды содержатся в регистрах *r* (*Rr*) и *d* (*Rd*).
Результат сохраняется в регистре *d* (*Rd*).



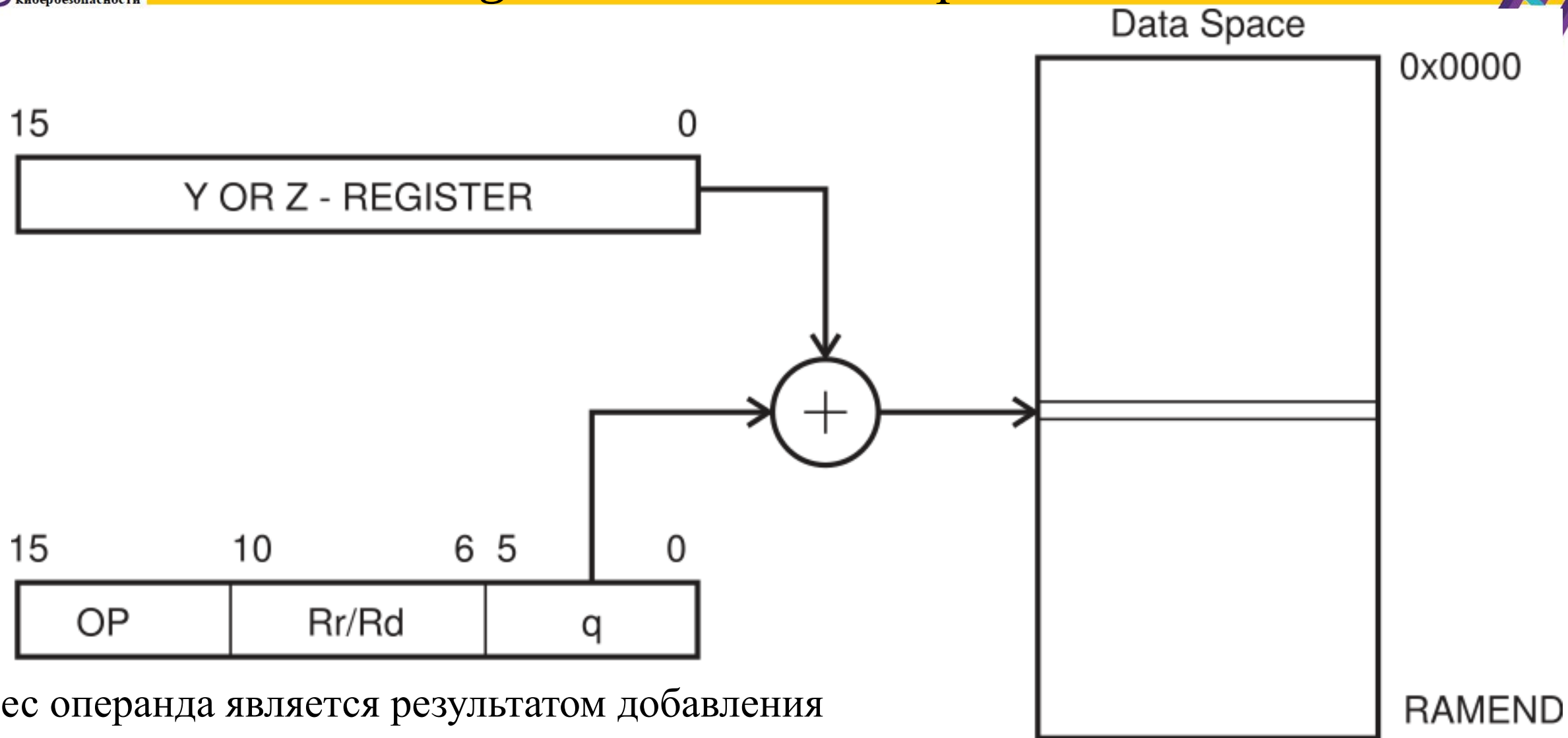
Адрес операнда содержится в шести битах командного слова.
Rr/Rd - адрес регистра назначения или источника.



16-битный адрес данных содержится в 16 младших битах двухсловной инструкции.
 Rd/Rr указывает регистр назначения или источника.

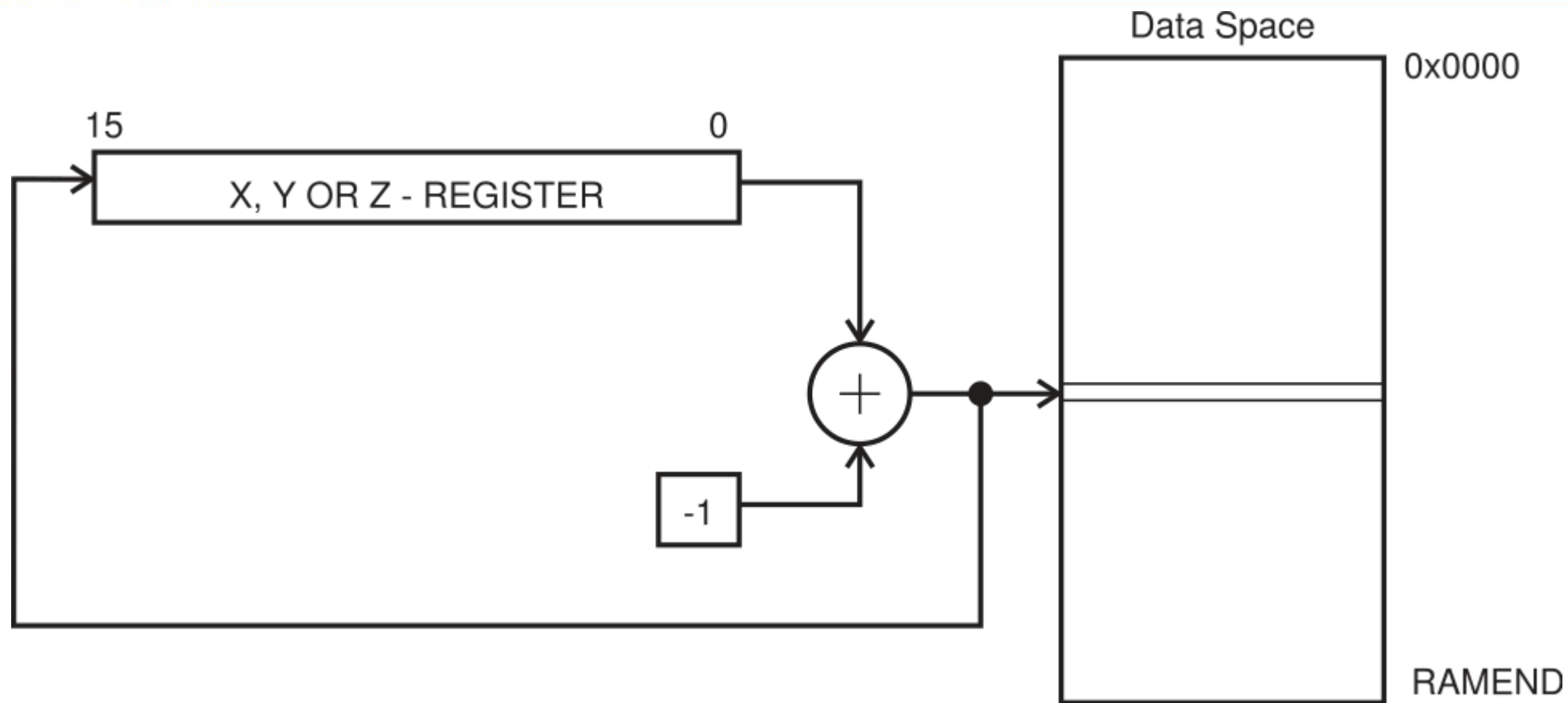


Адрес операнда - это содержимое X-, Y- или Z-регистра.



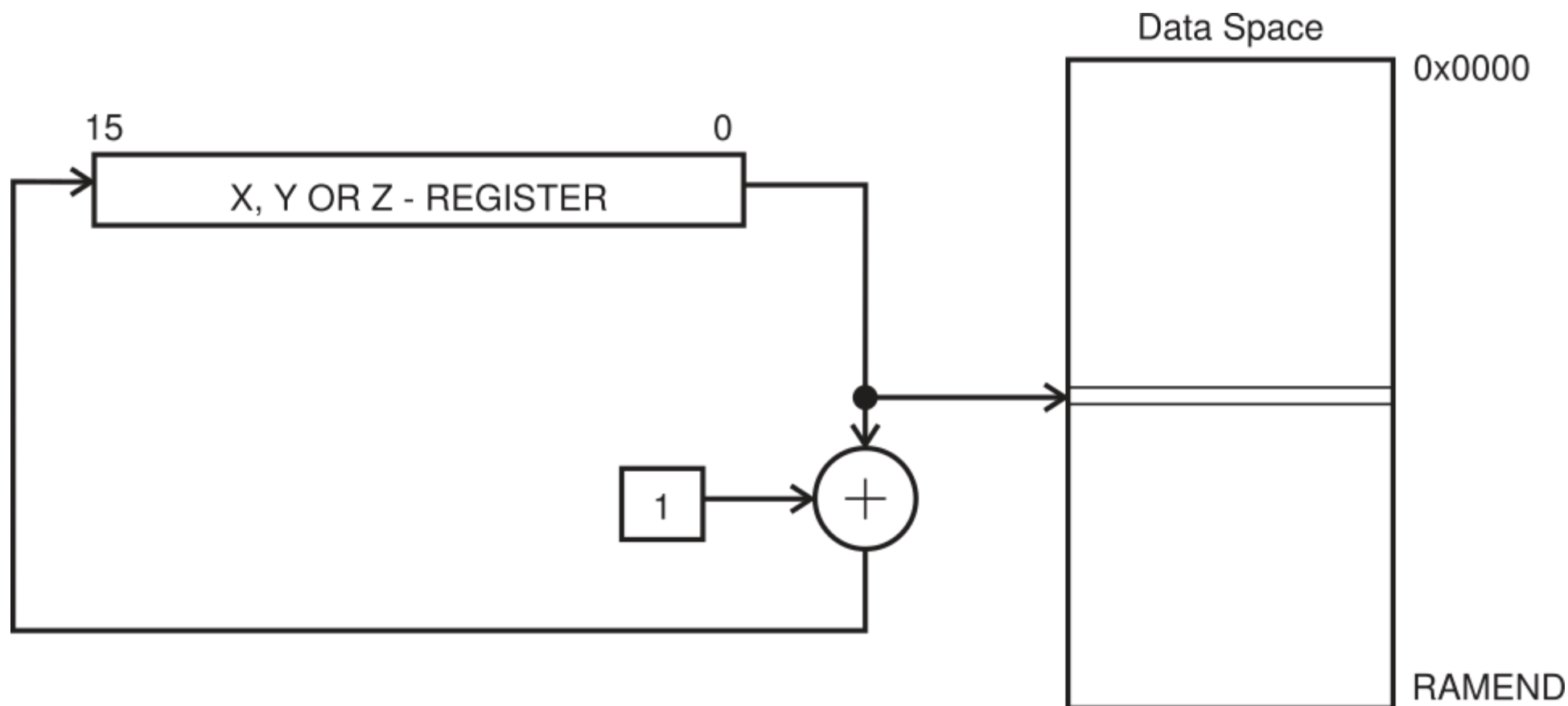
Адрес операнда является результатом добавления содержимого Y- или Z-регистра к адресу, содержащемуся в шести битах командного слова.

Rd/Rr указывает регистр назначения или источника.



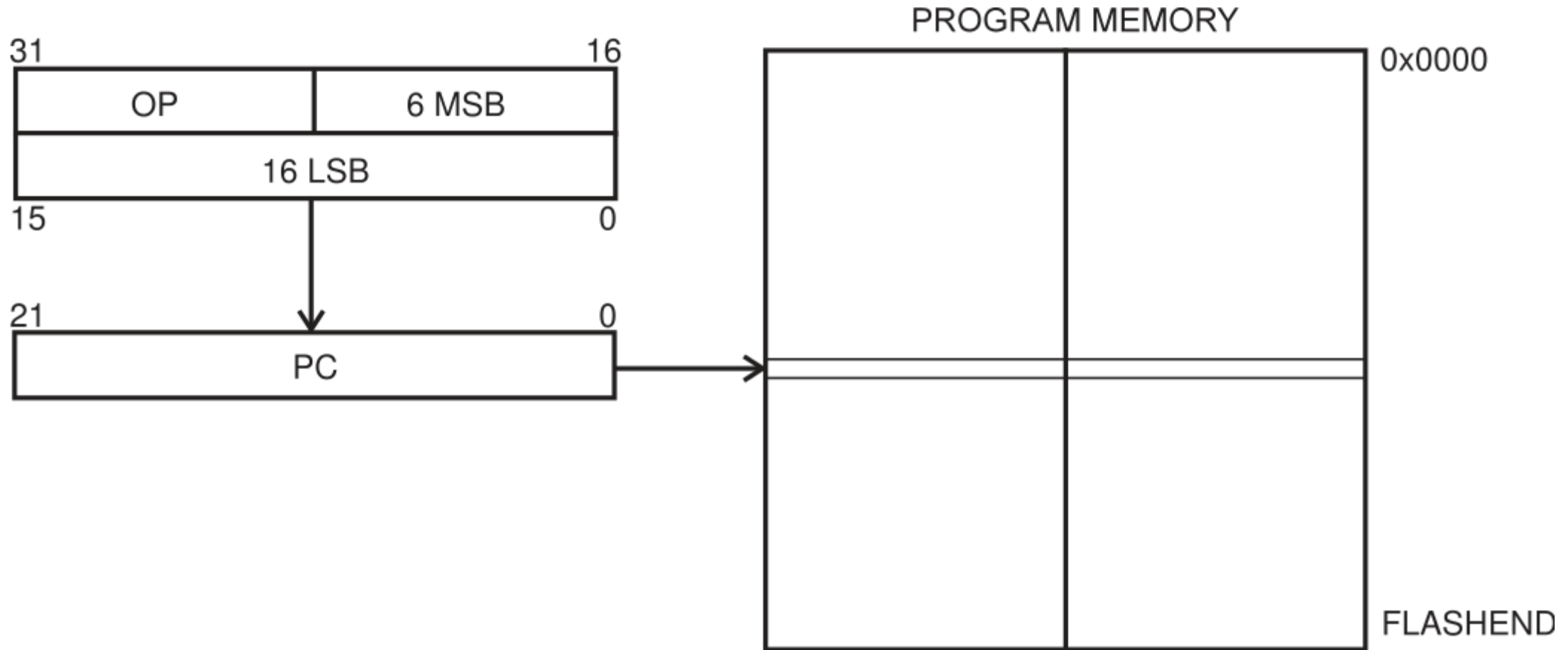
Перед операцией регистры X, Y или Z уменьшаются.

Адрес операнда - это уменьшенное содержимое X-, Y- или Z-регистра.

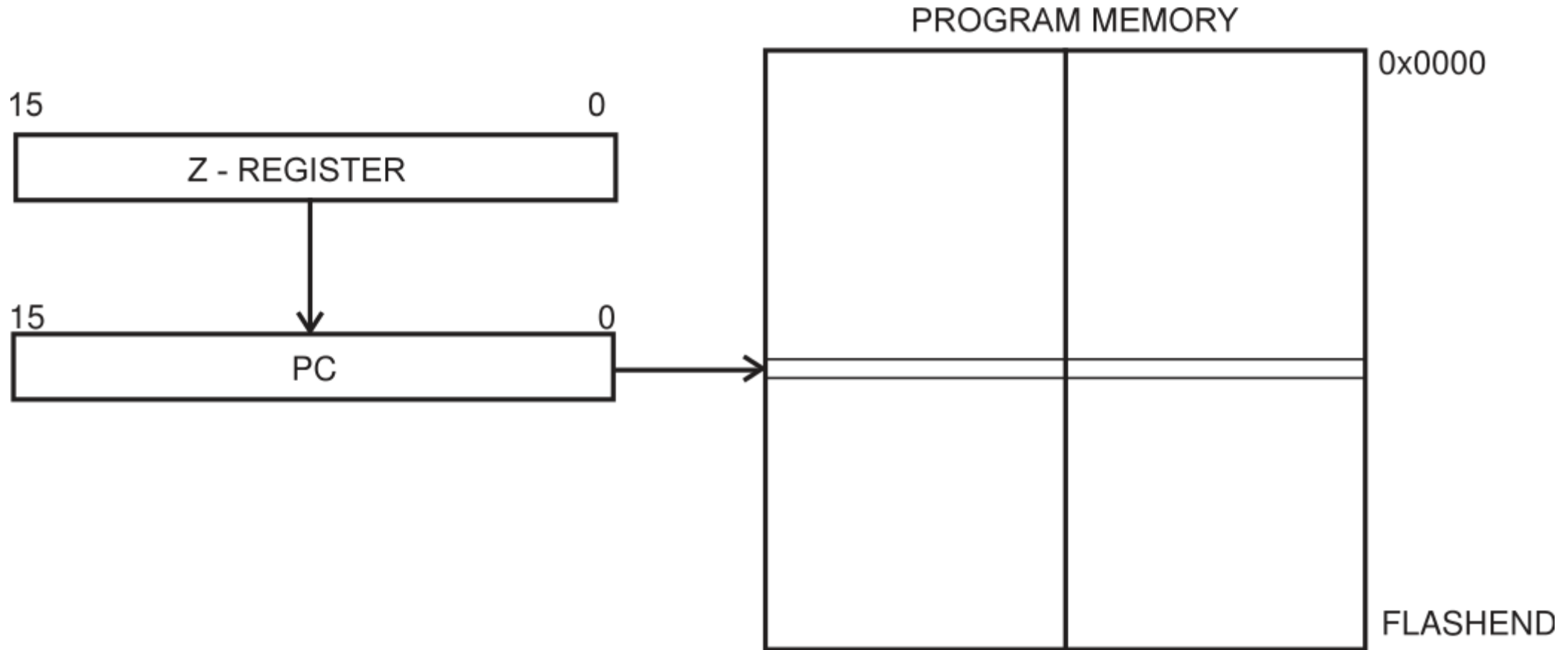


После операции увеличивается регистр X, Y или Z.

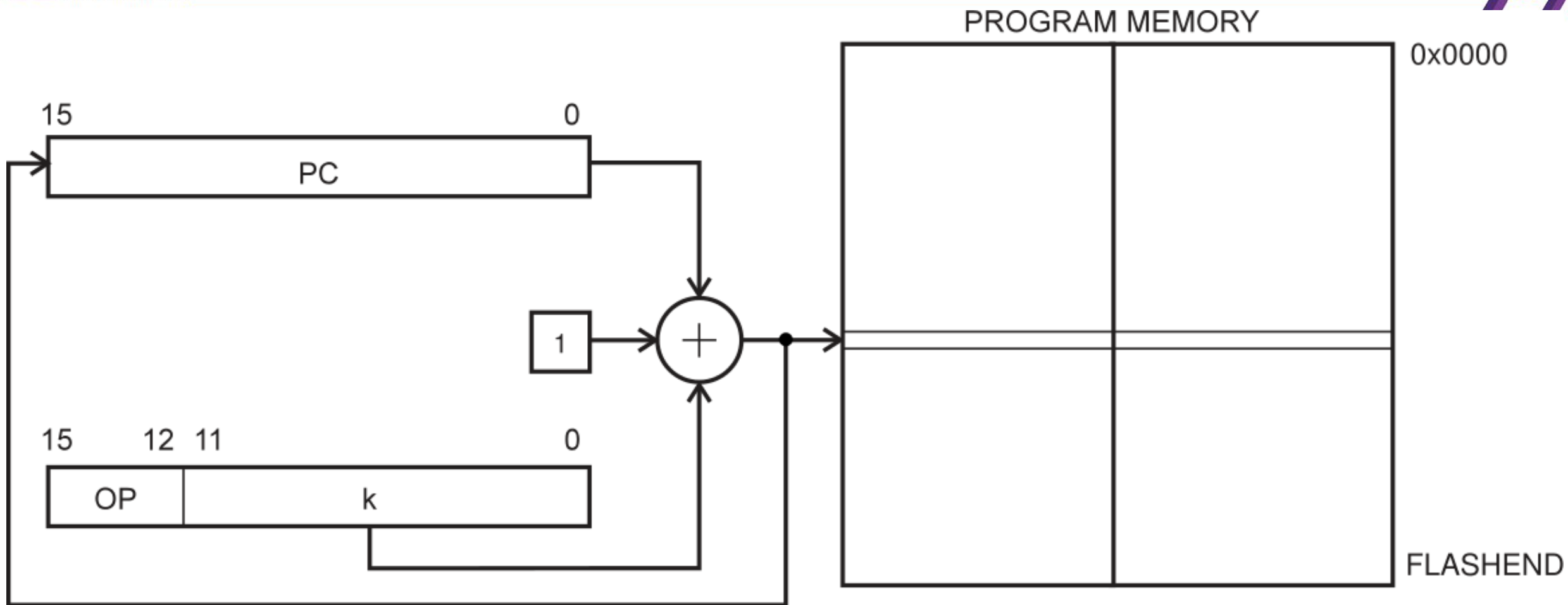
Адрес операнда - это содержимое X-, Y- или Z-регистра до приращения.



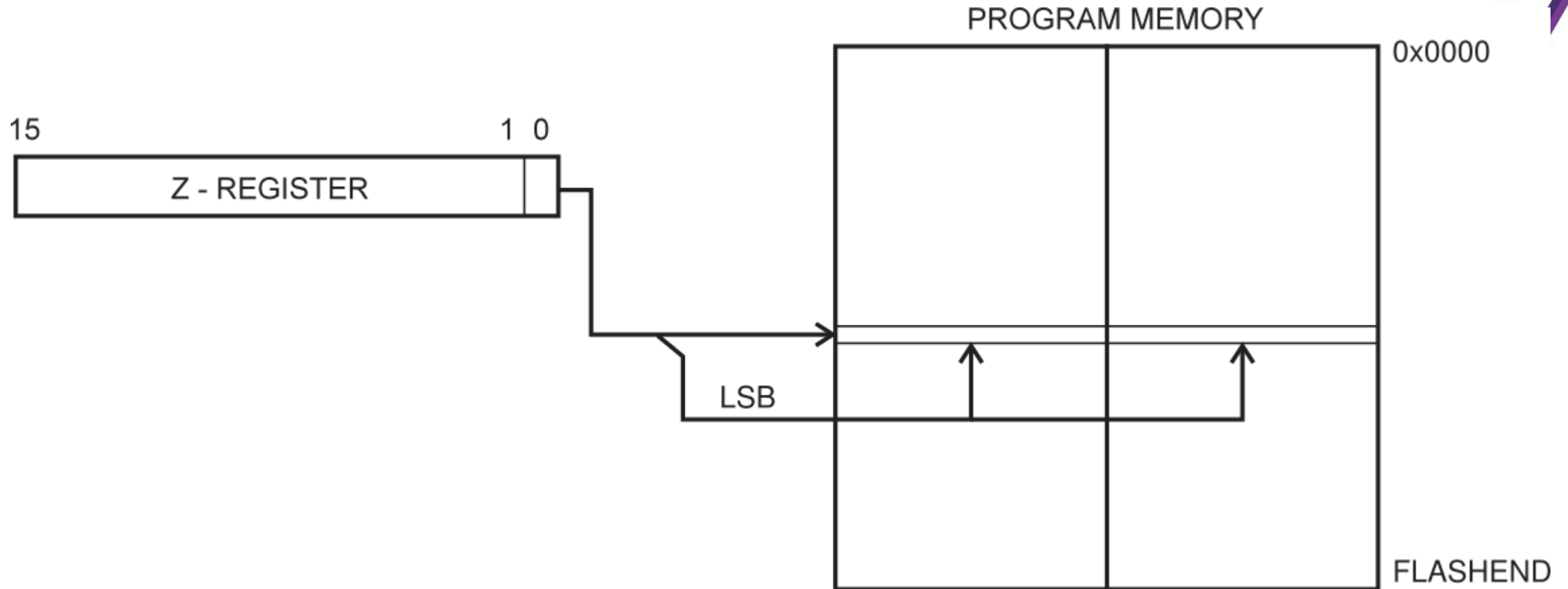
Выполнение программы продолжается по адресу, указанному непосредственно в командном слове.



Выполнение программы продолжается по адресу, содержащемуся в Z-регистре (т.е. в счётчик команд загружается содержимое Z-регистра).



Выполнение программы продолжается по адресу $PC+k+1$.
Относительный адрес k находится в диапазоне от -2048 до 2047.



Адрес байта определяется содержимым Z-регистра.
15 старших битов выбирают адрес слова.



Длина адресного пространства, адресность и разрядность:

- увеличение разрядности позволяет увеличить адресность команды и длину адреса (т.е. объем памяти, доступной команде);
- увеличение адресности приводит к повышению быстродействия обработки (за счёт снижения числа требуемых команд).

Например, выполнение сложения двух чисел (извлечь число по A1, число по A2, сложить и записать результат по A3):

- для трёхадресной машины — одна команда;
- для двухадресной — две команды;
- для одноадресной — три команды.



- Реальный режим (16-битный)
 - ☐ доступно 1Мбайт памяти
 - ☐ сегментная адресация (параграф – 16 байт, страница – 256 байт, сегмент – 64Кбайта)
- Защищённый режим (32-битный)
 - ☐ доступно 4 Гбайт памяти (x86)
 - ☐ защищённая память
 - ☐ виртуальная память
 - ☐ многозадачность
- Виртуальный режим
 - ☐ адресная модель реального режима
 - ☐ программы выполняются в наименее привилегированном кольце памяти



➤ Строки:

- ❑ ASCII

- ❑ Строка – последовательность байт – неперевернутый вид

- ❑ Строка – последовательность слов – перевёрнутый вид каждого элемента

➤ Адрес:

- ❑ 16-битовое смещение

- ❑ 20-битовый адрес (сегмент: смещение):

$$\text{Адрес} = \text{сегмент} * 16 + \text{смещение}$$



- Длина команд 1-6 байтов
- КОП – 1-2 первых байта
- Операнды – 0-3 байт; размер – байт, слово, двойное слово
- Модификаторы адреса:

$A, A[M], A[M1][M2]$

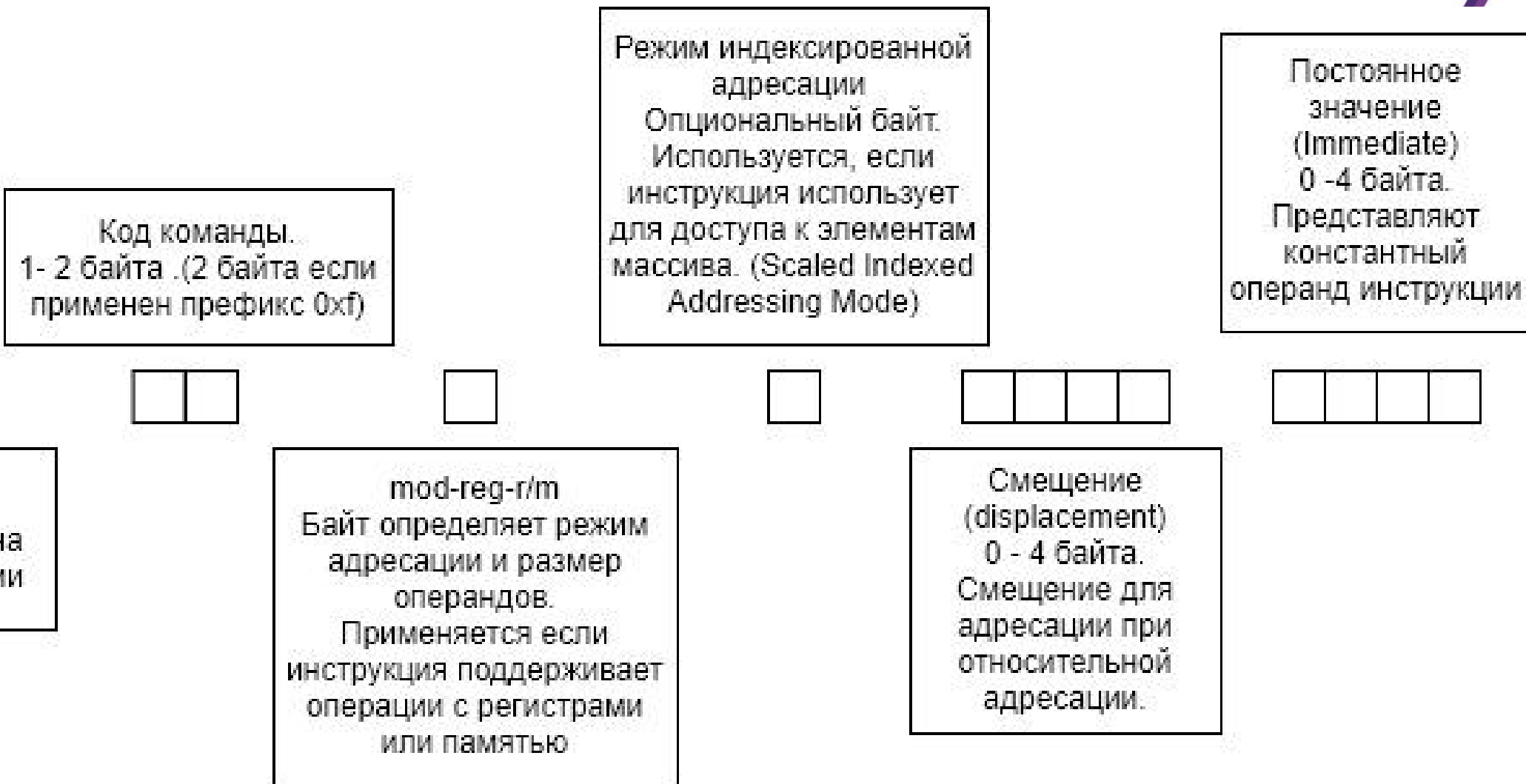
M – BX, BP, SI или DI

$M1$ – BX или BP

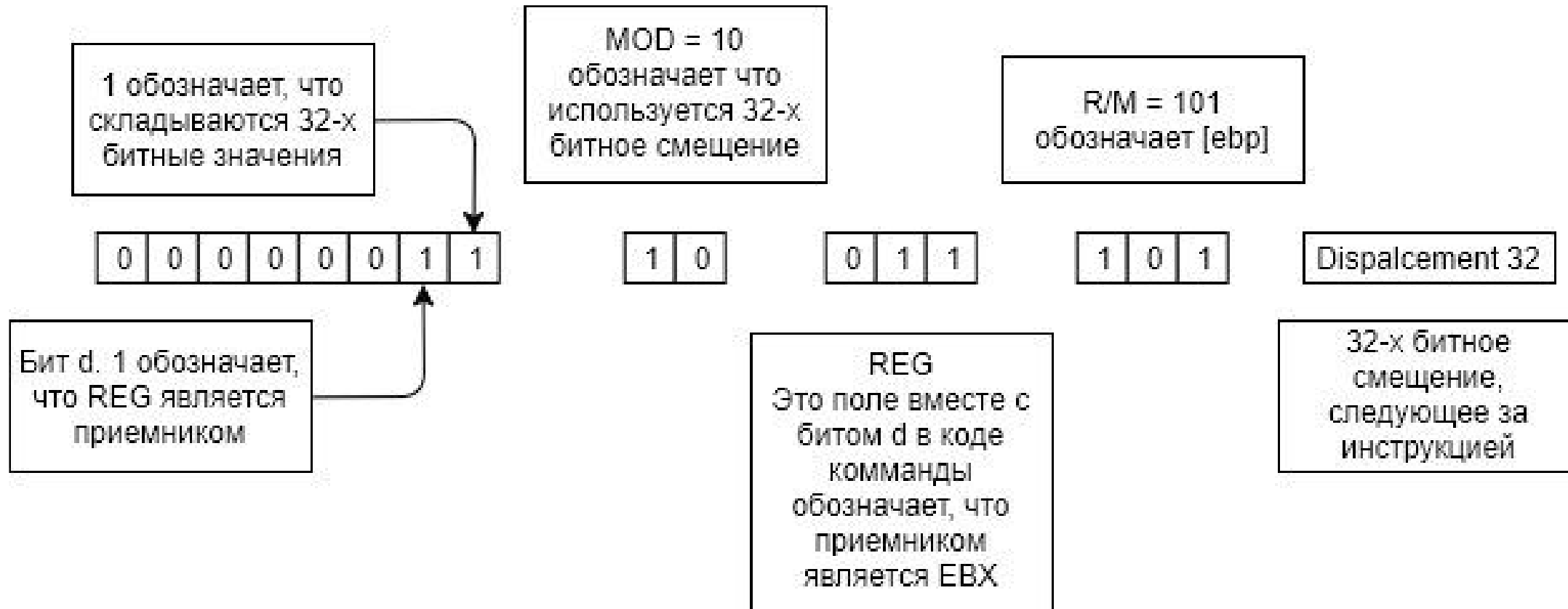
$M2$ – SI или DI

❑ $A_{\text{исп}} = (A + [M]) \bmod 2^{16}$

❑ $A_{\text{исп}} = (A + [M1] + [M2]) \bmod 2^{16}$



`add ebx, [ ebp + disp32 ]`



`add ebx, [ ebp + disp32 ] = 03, 9D, ww, xx, yy, zz`

байты `ww`, `xx`, `yy`, `zz` описывают смещение,
от младшего байта `ww` к старшему байту `zz`

Формат команды «регистр – регистр»

КОП		d	w	11		reg1		reg2	
7	2	1	0	7	6	5	3	2	0

w = 1 = 0	— слова	d = 1 = 0	reg1 := reg1 - reg2
	— байты		reg2 := reg2 - reg1

reg	w = 1	w = 0
000	AX	AL
001	CX	CL
010	DX	DL
011	BX	BL
100	SP	AH
101	BP	CH
110	SI	DH
111	DI	BH



КОП		w	mod	reg	mem	Адрес (0—2 байта)
-----	--	---	-----	-----	-----	-------------------

mem	mod	00	01	10
000		$[BX] + [SI]$	$[BX] + [SI] + a8$	$[BX] + [SI] + a16$
001		$[BX] + [DI]$	$[BX] + [DI] + a8$	$[BX] + [DI] + a16$
010		$[BP] + [SI]$	$[BP] + [SI] + a8$	$[BP] + [SI] + a16$
011		$[BP] + [DI]$	$[BP] + [DI] + a8$	$[BP] + [DI] + a16$
100		$[SI]$	$[SI] + a8$	$[SI] + a16$
101		$[DI]$	$[DI] + a8$	$[DI] + a16$
110		$a16$	$[BP] + a8$	$[BP] + a16$
111		$[BX]$	$[BX] + a8$	$[BX] + a16$



КОП	s	w	11	КОП'	reg	Непосред. операнд (1—2 б.)
-----	---	---	----	------	-----	----------------------------

КОП	s	w	mod	КОП"	mem	Адрес (0—2 б.)	Непоср. оп (1—2 б.)
-----	---	---	-----	------	-----	----------------	---------------------



- *Внутрисегментный относительный короткий переход:*
JMP i8 ($IP := IP + i8$)
- *Внутрисегментный относительный длинный переход:*
JMP i16 ($IP := IP + i16$) ; (JMP SHORT L)
- *Внутрисегментный абсолютный косвенный переход:*
JMP r16 ($IP := [r]$) или JMP m16 ($IP := [m16]$)
- *Межсегментный абсолютный прямой переход:*
JMP seg:ofs ($CS := seg, IP := ofs$)
- *Межсегментный абсолютный косвенный переход:*
JMP m32 ($CS := [m32 + 2], IP := [m32]$)

MOV AX, 0 ; начальное значение суммы
MOV SI, 0 ; начальное значение индексного регистра
MOV CX, 10; число повторений цикла

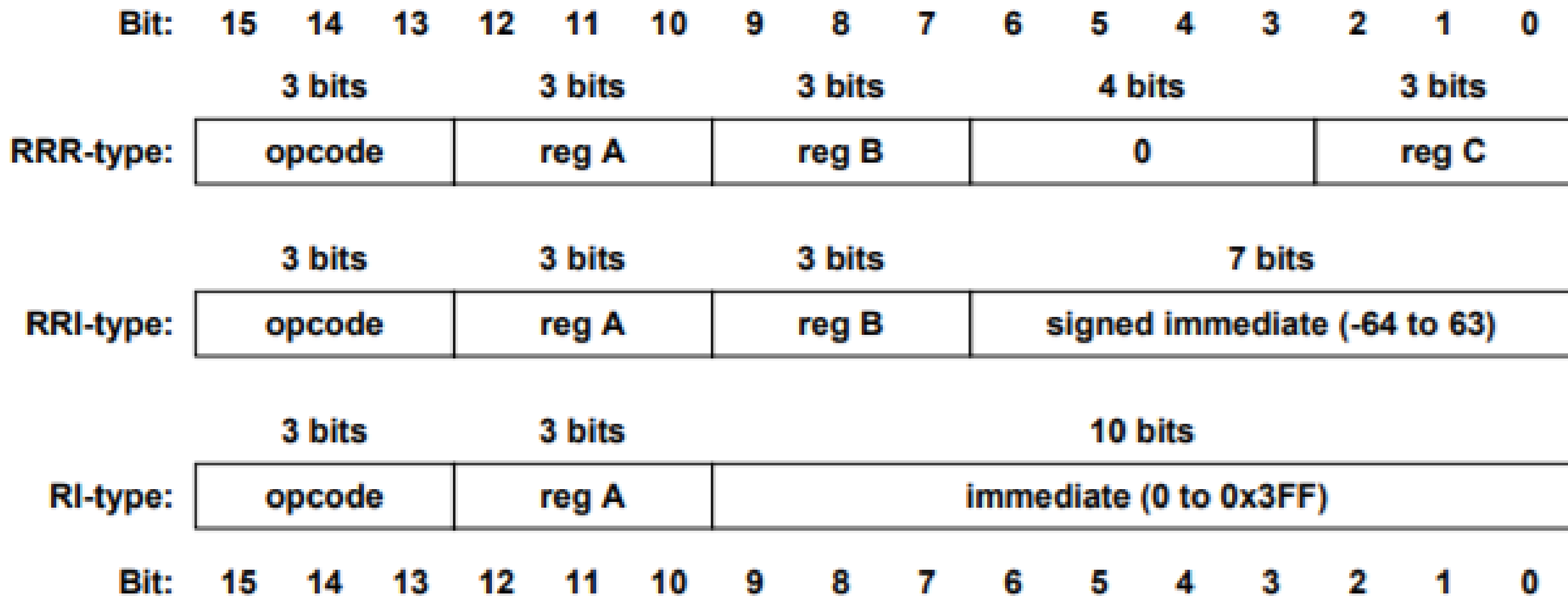
L:

ADD AX, X[SI] ; $AX := AX + X[i]$
ADD SI, 2 ; $SI := SI + 2$
LOOP L ; $CX := CX - 1$; if $(CX \neq 0)$ goto L
MOV S, AX ; $S := AX$



- команда загрузки элемента строки в аккумулятор (*lodsb* или *lodsw*);
- запись аккумулятора в строку (*stosb* или *stosw*);
- пересылка строк (*movsb* или *movsw*);
- сравнение строк (*cmpsb* или *cmpsw*);
- сканирование строки (*scasb* или *scasw*).

```
CLD                ; DF:=0 (просмотр строки вперед)
MOV CX,1000        ; CX — число повторений
MOV AX,DS
MOV ES,AX          ; ES:=DS
LEA SI,A           ; ES:SI — «откуда»
LEA DI,B           ; DS:DI — «куда»
REP MOVSB          ; пересылка CX байтов.
```



Примеры команд RISC-16

	3 bits			3 bits			3 bits			4 bits				3 bits		
ADD:	000			reg A			reg B			0				reg C		
	3 bits			3 bits			3 bits			7 bits						
ADDI:	001			reg A			reg B			signed immediate (-64 to 63)						
	3 bits			3 bits			3 bits			4 bits				3 bits		
NAND:	010			reg A			reg B			0				reg C		
	3 bits			3 bits			10 bits									
LUI:	011			reg A			immediate (0 to 0x3FF)									
	3 bits			3 bits			3 bits			7 bits						
SW:	100			reg A			reg B			signed immediate (-64 to 63)						
	3 bits			3 bits			3 bits			7 bits						
LW:	101			reg A			reg B			signed immediate (-64 to 63)						
	3 bits			3 bits			3 bits			7 bits						
BEQ:	110			reg A			reg B			signed immediate (-64 to 63)						
	3 bits			3 bits			3 bits			7 bits						
JALR:	111			reg A			reg B			0						
Bit:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0



	15						0	
1	00cc	ccrd	dddd	rrrr	АЛУ: Opcode(c) Rd, Rr			
2	1001	010d	dddd	cccc	Расширенный формат АЛУ: Opcode(c) Rd			
3	01cc	KKKK	dddd	KKKK	АЛУ + непосредственный операнд: Opcode(c) Rd, #K			
4	10Q0	QQcd	dddd	cQQQ	Загрузка/сохранение: ld/st(c) X/Y/Z+Q, Rd			
5	11cc	KKKK	KKKK	KKKK	Переход: br(c) PC + K			
	31							0
6	1001	010K	KKKK	11cK	KKKK	KKKK	KKKK	KKKK

Формат команд ATmega32



№	Машинный код для формата команд																Описание			
1	1	0	0	1	.	0	0	n	r	.	r	r	r	r	.	0	0	0	0	Передача данных с прямой адресацией
	k	k	k	k	.	k	k	k	k	.	k	k	k	k	.	k	k	k	k	
2	1	0	0	1	.	0	1	0	k	.	k	k	k	k	.	1	1	n	k	Безусловные с прямой адресацией
	k	k	k	k	.	k	k	k	k	.	k	k	k	k	.	k	k	k	k	
3	1	1	0	n	.	k	k	k	k	.	k	k	k	k	.	k	k	k	k	Безусловные с относительной адресацией
4	1	0	0	1	.	0	1	0	n	.	0	0	0	n	.	1	0	0	n	Безусловные с косвенно-регистровой адресацией
5	1	1	1	1	.	0	n	k	k	.	k	k	k	k	.	k	s	s	s	Условные с относительной адресацией
6	0	0	0	0	.	0	0	0	0	.	0	0	0	0	.	0	0	0	0	Команда NOP
	1	0	0	1	.	0	1	0	1	.	1	0	n	n	.	1	0	0	0	Управляющие команды
7	1	0	0	1	.	0	1	0	0	.	n	s	s	s	.	1	0	0	0	Команды установки флагов в регистре SREG
8	0	0	n	n	.	n	n	r	d	.	d	d	d	d	.	r	r	r	r	Бинарные арифметические и логические команды
	1	0	0	1	.	1	1	r	d	.	d	d	d	d	.	r	r	r	r	Беззнаковое умножение
9	0	0	0	0	.	0	0	0	1	.	d	d	d	d	.	r	r	r	r	Пересылка регистровой пары
	0	0	0	0	.	0	0	1	0	.	d	d	d	d	.	r	r	r	r	Умножение чисел со знаком
10	0	0	0	0	.	0	0	1	1	.	n	d	d	d	.	n	r	r	r	Умножение смешанных чисел и дробное умножение
11	1	0	1	1	.	n	A	A	r	.	r	r	r	r	.	A	A	A	A	Команды установки/считывания всех разрядов портов I/O
12	1	0	0	1	.	1	0	n	n	.	A	A	A	A	.	A	b	b	b	Команды установки/считывания/переходов по разрядам портов I/O
13	1	1	1	1	.	1	n	n	r	.	r	r	r	r	.	0	b	b	b	Битовые команды
14	1	0	q	0	.	q	q	n	r	.	r	r	r	r	.	n	q	q	q	Передача данных с косвенно-регистровой адресацией и смещением
15	0	1	n	n	.	k	k	k	k	.	r	r	r	r	.	k	k	k	k	Унарные команды с операндом
16	1	0	0	1	.	0	0	n	r	.	r	r	r	r	.	n	n	n	n	Передача данных с косвенно-регистровой адресацией
	1	0	0	1	.	0	1	0	r	.	r	r	r	r	.	n	n	n	n	Унарные команды без операнда
17	1	0	0	1	.	0	1	1	n	.	k	k	r	r	.	k	k	k	k	Арифметические команды с регистровыми параметрами